

ERAMIA-RS
II ESCOLA REGIONAL DE APRENDIZADO DE MÁQUINA E INTELIGÊNCIA ARTIFICIAL DA REGIÃO SUL
2026

Aprendizado por Reforço

dos Fundamentos à Abstração Temporal

Gabriel de O. Ramos
Arturo de Souza
Augusto A. F. Leal

UNISINOS

© 2026 by Gabriel de Oliveira Ramos under CC BY-NC-ND

Quem somos



Gabriel Ramos
Professor



Arturo de Souza
Doutorando

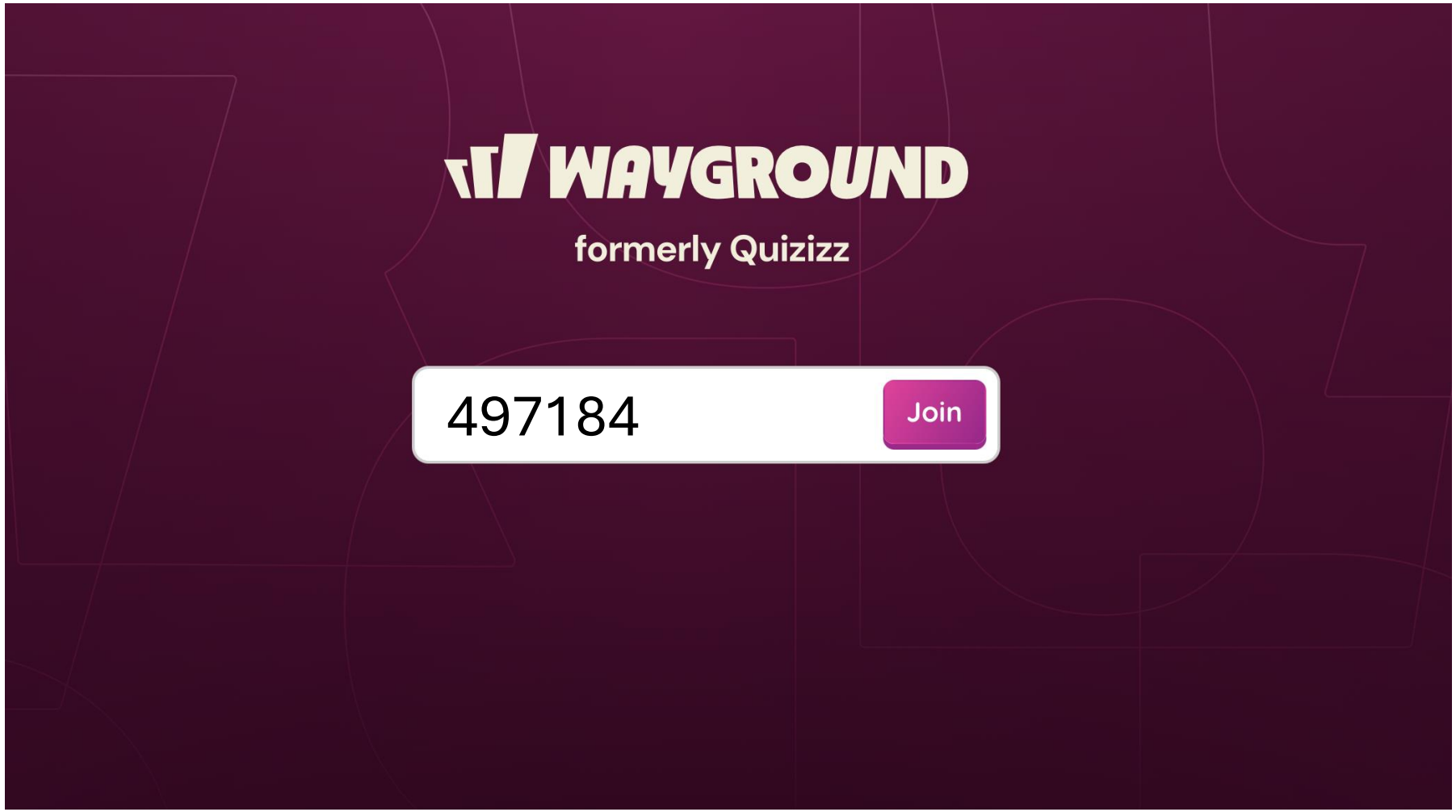


Augusto Leal
Doutorando

Agenda

- 08:30-10:00 Parte 1:
  - Abertura (5min)
  - Contexto (5min)
  - Fundamentos (15min)
  - Algoritmos (20min)
  - Prática (30min)
  - Aplicações (10min)
- 10:00-11:00 Coffee break + pôsteres
- 11:00-12:00 Parte 2:
  - Options (20min)
  - Prática (30min)
  - Encerramento (10min)

Competição



https://wayground.com/join

Contexto

Aprendizado de Máquina

- Agente melhora seu **desempenho em tarefas futuras** com base em **experiências e observações do mundo**

Aprendizado de Máquina

- Agente melhora seu **desempenho em tarefas futuras** com base em **experiências e observações do mundo**



Aprendizado de Máquina

- Agente melhora seu **desempenho em tarefas futuras** com base em **experiências e observações do mundo**





Aprendizado de Máquina

- Agente melhora seu **desempenho em tarefas futuras** com base em **experiências e observações do mundo**







Aprendizado de Máquina

- Agente melhora seu **desempenho em tarefas futuras** com base em **experiências e observações do mundo**

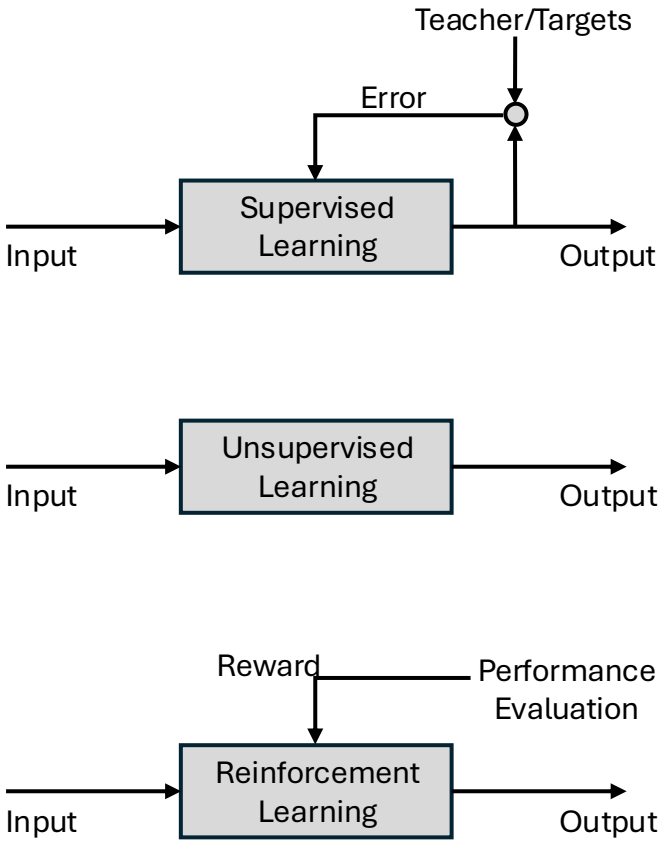






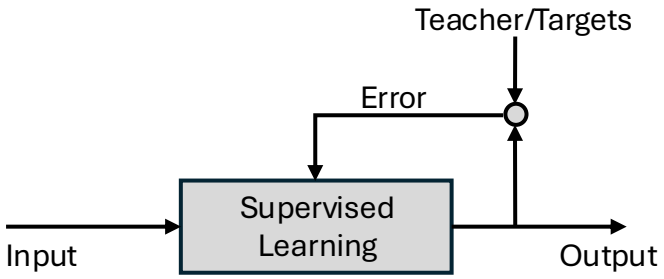
Tipos de aprendizado

- Aprendizado **supervisionado**:
  - Aprender uma função capaz de mapear entradas para saídas a partir de um conjunto de exemplos de entradas e saídas
- Aprendizado **não-supervisionado**:
  - Aprender padrões em conjuntos de dados não-rotulados
- Aprendizado **por reforço**:
  - Aprender um comportamento através de recompensas eventuais (recebidas ao interagir com o ambiente) para atingir determinado objetivo
  - Tentativa e erro



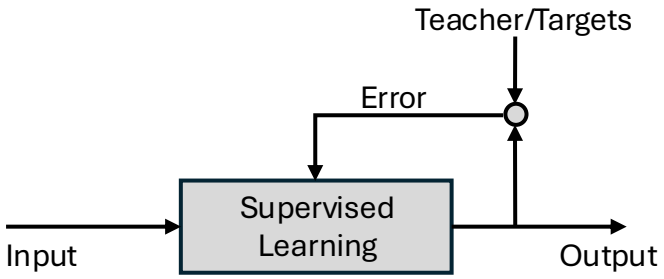
Tipos de aprendizado

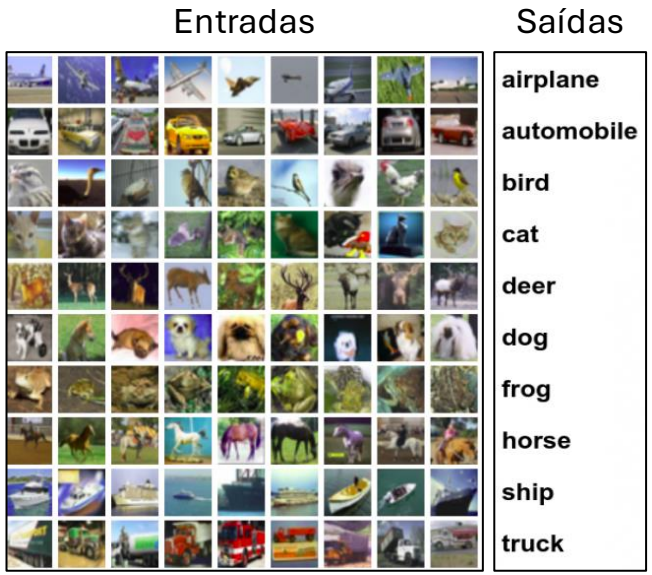
- Aprendizado **supervisionado**:
  - Aprender uma função capaz de mapear entradas para saídas a partir de um conjunto de exemplos de entradas e saídas

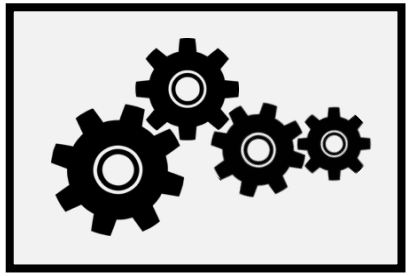


Tipos de aprendizado

- Aprendizado **supervisionado**:
  - Aprender uma <u>função</u> capaz de mapear entradas para saídas a partir de um conjunto de exemplos de <u>entradas e saídas</u>

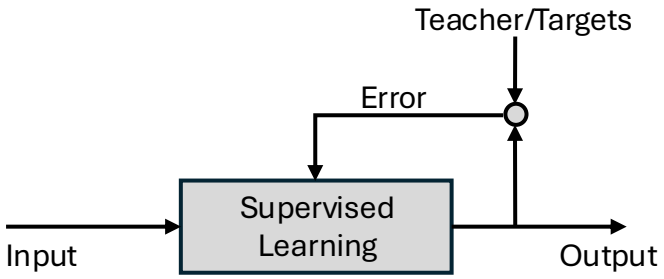


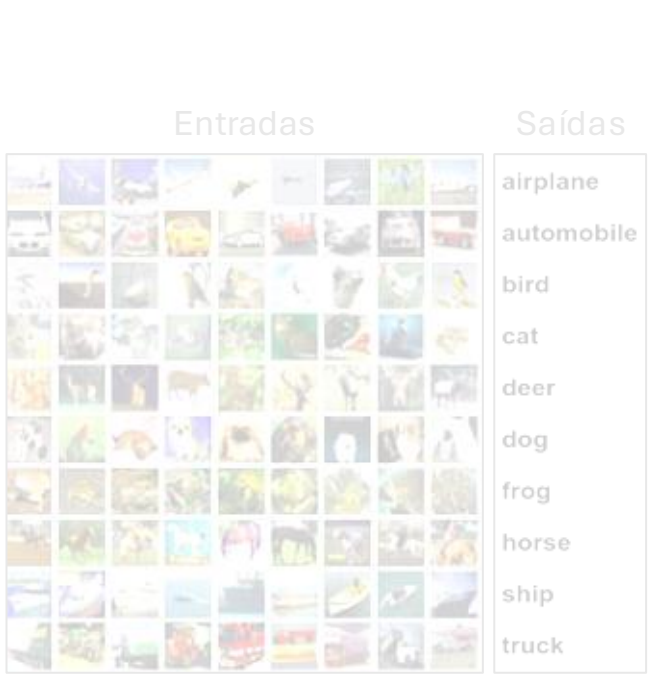


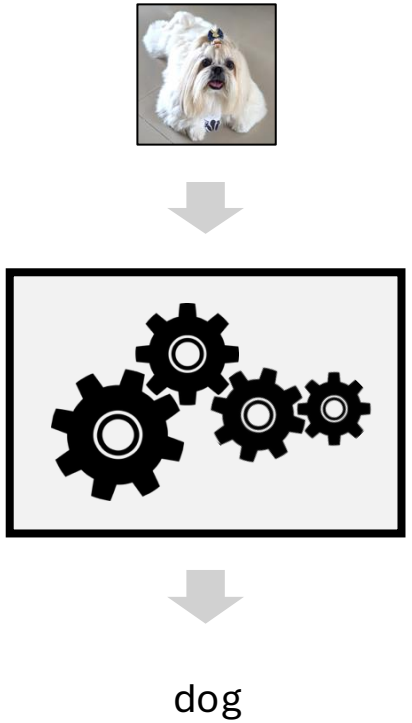


Tipos de aprendizado

- Aprendizado **supervisionado**:
  - Aprender uma função capaz de mapear entradas para saídas a partir de um conjunto de exemplos de entradas e saídas







Mas...

Quiz

Quais são possíveis limitações do aprendizado supervisionado e não supervisionado, especialmente em comparação ao aprendizado por reforço?

Mas...

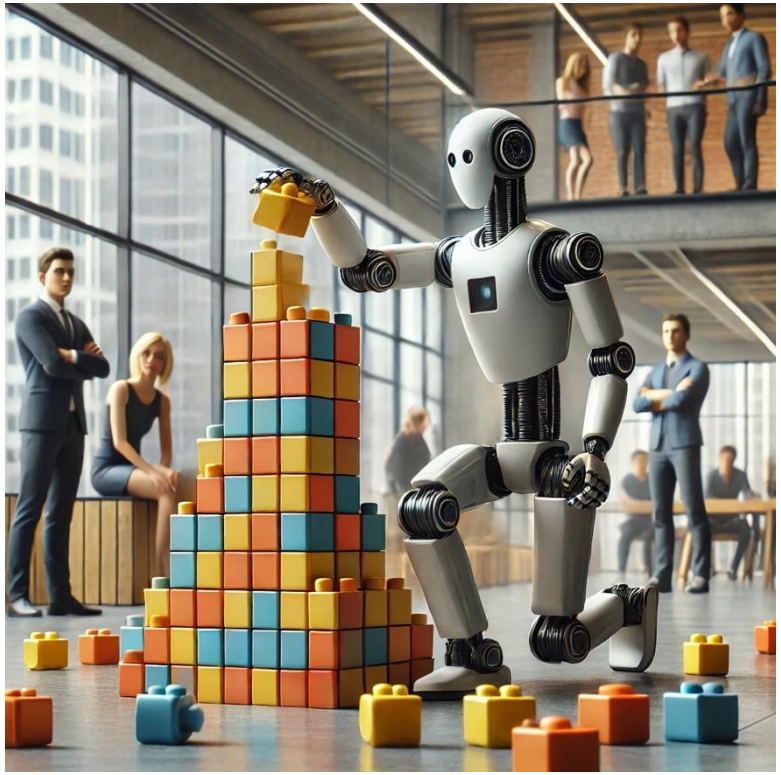
- Dependem de dados **previamente coletados**



Mas...

- Dependem de dados **previamente coletados**

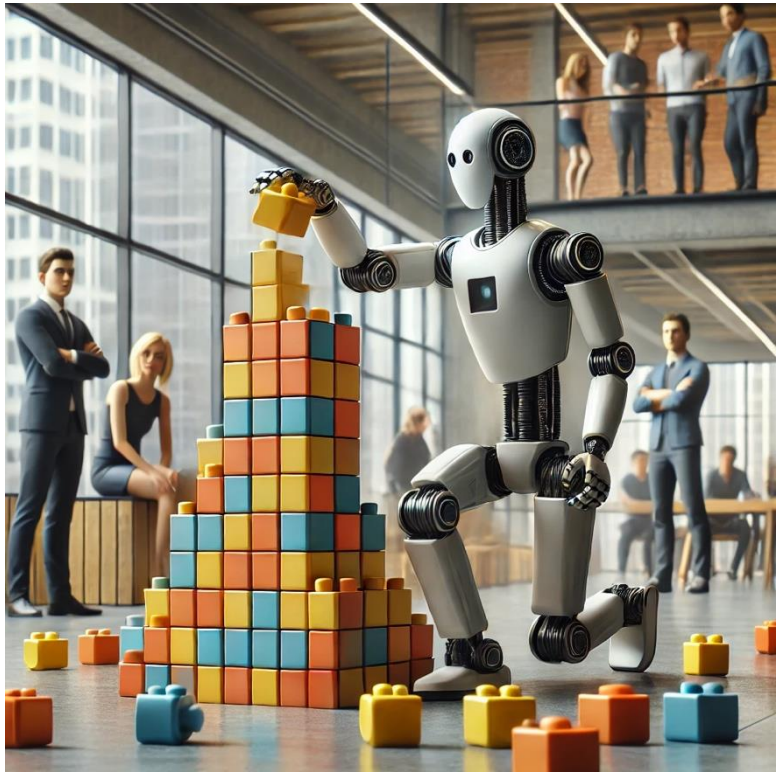




Mas...

- Dependem de dados **previamente coletados**
- Resolvem subproblemas sem saber como isso se encaixa no todo

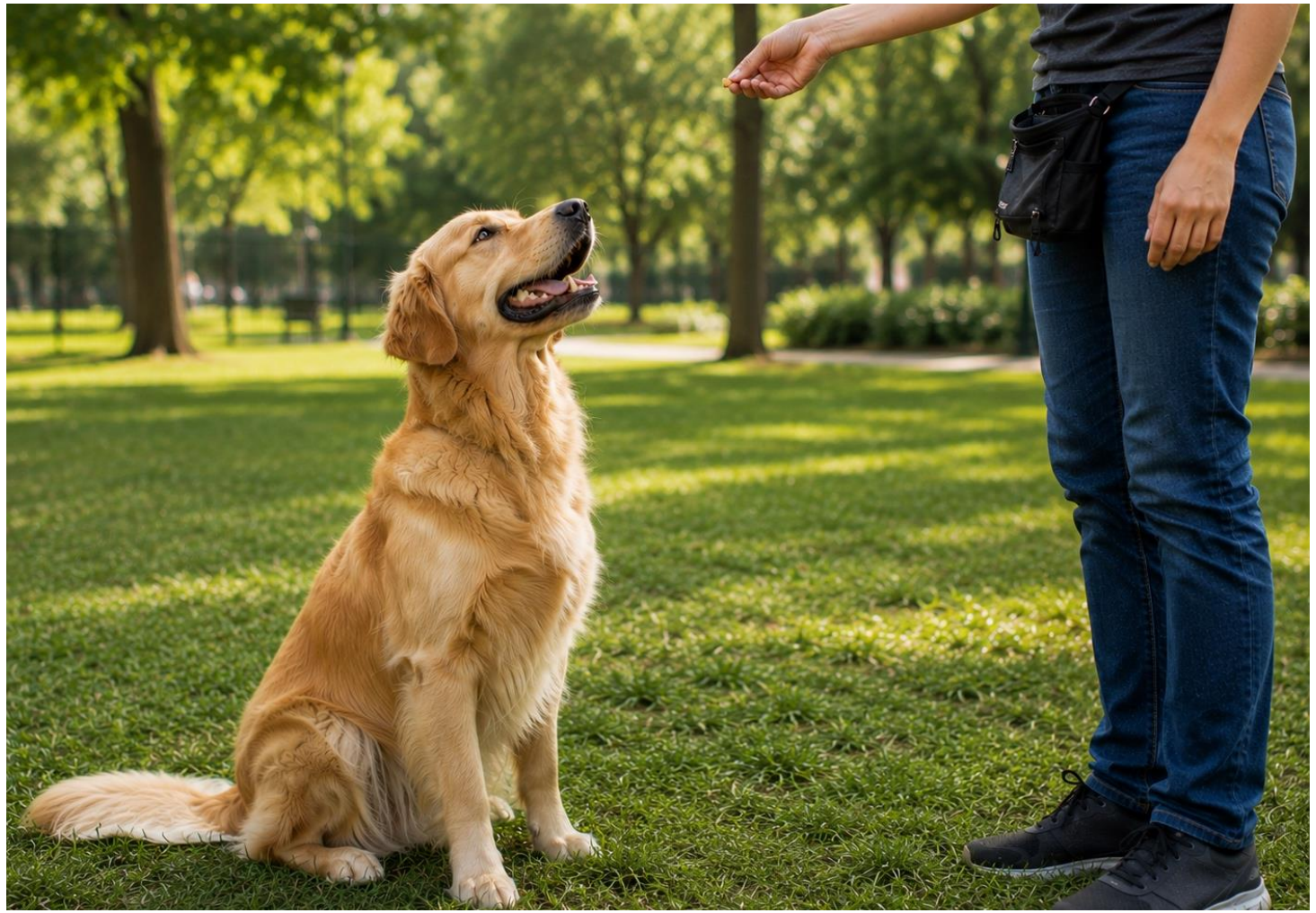




Aprendizado por reforço

Motivação

- Aprender um **comportamento** através de tentativa e erro (a partir de interações com o ambiente) visando atingir um **objetivo**



Aprendizado por reforço

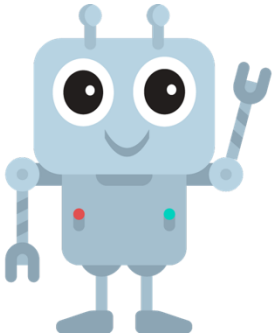
- Agente aprende através de **recompensas**/**punições**



https://www.youtube.com/watch?v=mDntbGRPeEU

Aprendizado por reforço

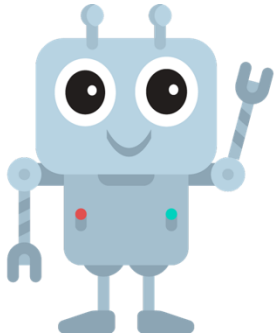
- **Agente**



Aprendizado por reforço

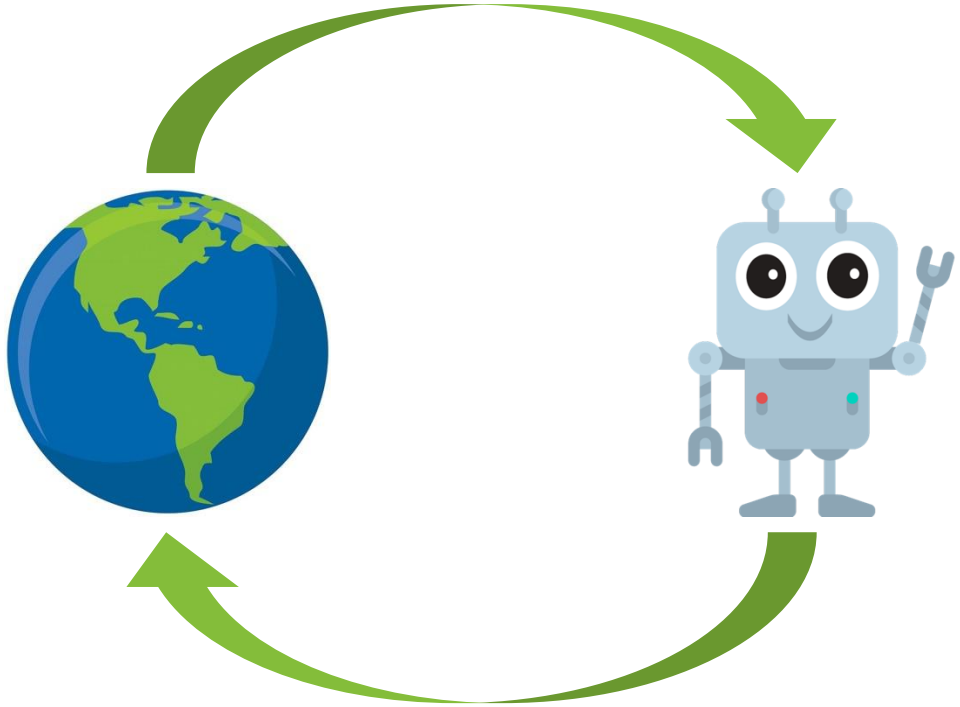
- **Agente** situado em um ambiente





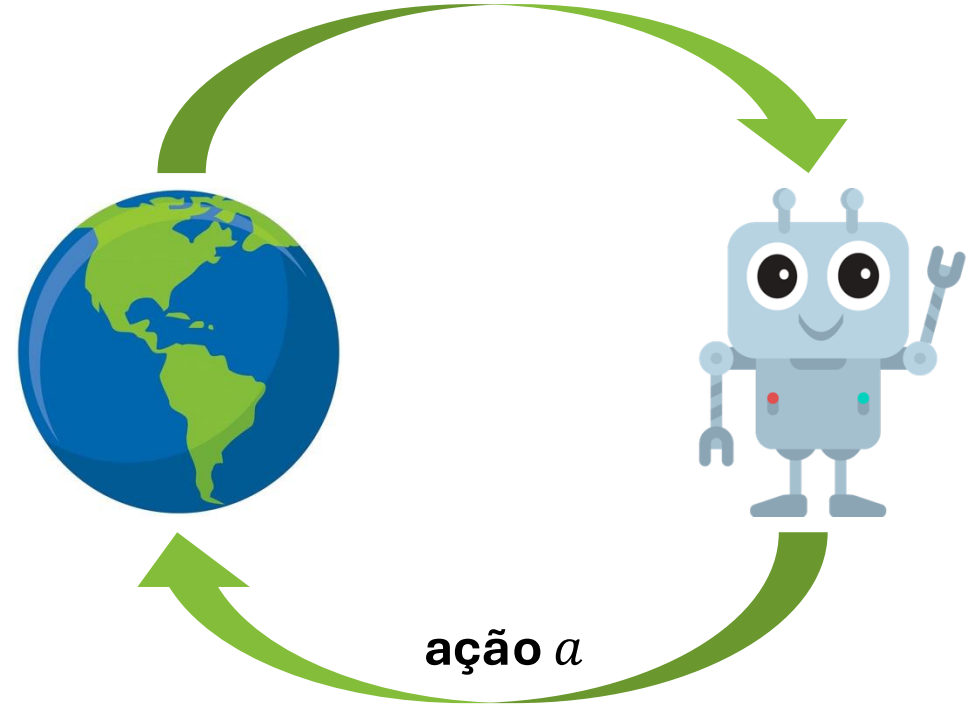
Aprendizado por reforço

- **Agente** situado em um ambiente
- Interação com o ambiente



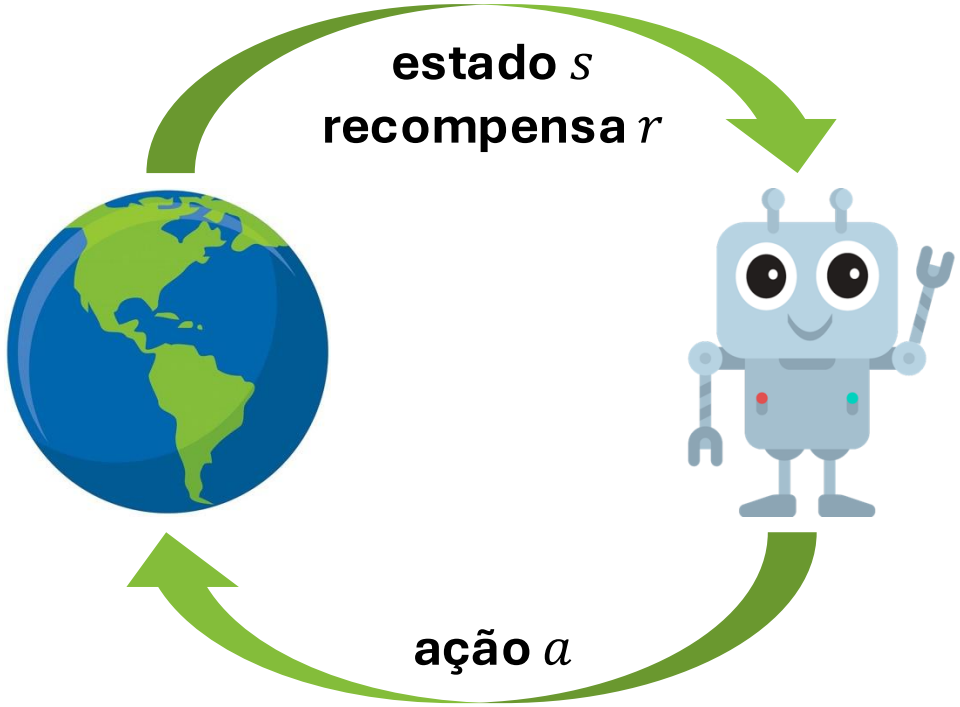
Aprendizado por reforço

- **Agente** situado em um ambiente
- Interação com o ambiente
  - Agente executa **ações**



Aprendizado por reforço

- **Agente** situado em um ambiente
- Interação com o ambiente
  - Agente executa **ações**
  - Ambiente fornece feedback avaliativo (**recompensas**)



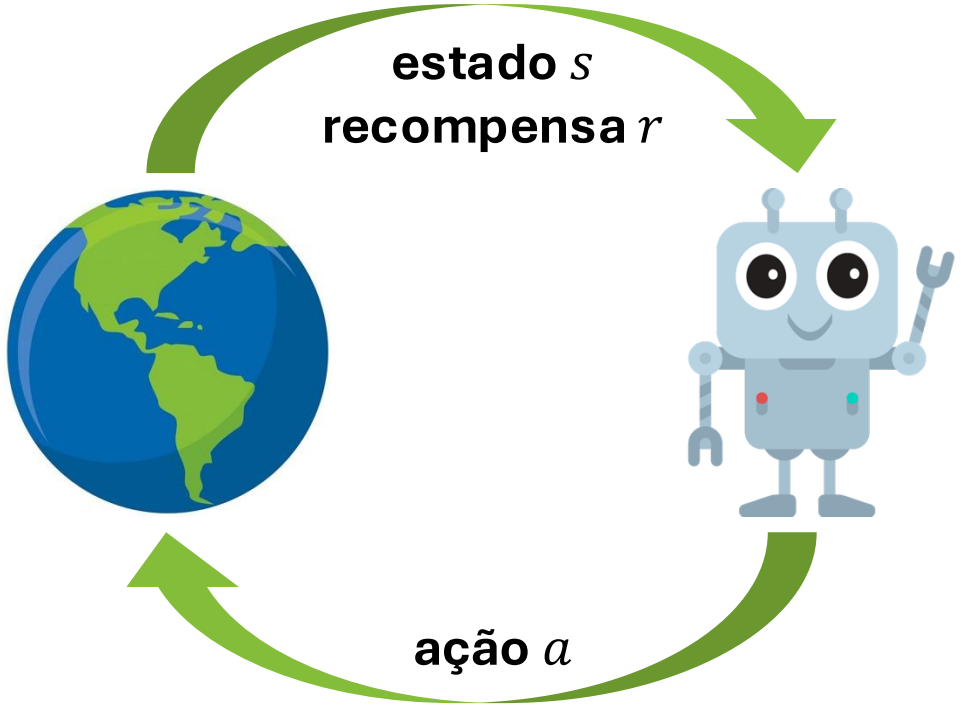
Aprendizado por reforço

- **Agente** situado em um ambiente
- Interação com o ambiente
  - Agente executa **ações**
  - Ambiente fornece feedback avaliativo (**recompensas**)



Aprendizado por reforço

- **Agente** situado em um ambiente
- Interação com o ambiente
  - Agente executa **ações**
  - Ambiente fornece feedback avaliativo (**recompensas**)
- Processo de decisão **sequencial**
  - Ações praticadas no presente podem afetar feedback futuro



(Alguns) Fundamentos

(Alguns) Fundamentos



https://research.google/blog/indoorsim-to-outdoorreal-learning-to-navigate-outdoors-without-any-outdoor-experience

(Alguns) Fundamentos

Estado s_t: percepção do agente sobre o ambiente no tempo atual



https://research.google/blog/indoorsim-to-outdoorreal-learning-to-navigate-outdoors-without-any-outdoor-experience

Recompensa r_t: sinal de feedback fornecido pelo ambiente

Quiz

Considere um robô equipado apenas com um sensor LiDAR. Quais das seguintes informações ele consegue obter diretamente desse sensor?

(Alguns) Fundamentos

Política π: o que fazer em cada estado

Funções de valor V_π **e** Q_π: retorno esperado ao usar a política π



https://www.youtube.com/watch?v=a6bim_76Omc

(Alguns) Fundamentos

- **Markov decision process (MDP)**
  - Propriedade de Markov: o estado atual fornece toda informação relevante sobre o ambiente para a tomada de decisão do agente

(Alguns) Fundamentos

- **Markov decision process (MDP)**
  - Propriedade de Markov: o estado atual fornece toda informação relevante sobre o ambiente para a tomada de decisão do agente



Exemplo

(Alguns) Fundamentos

- **Markov decision process (MDP)**
  - Propriedade de Markov: o estado atual fornece toda informação relevante sobre o ambiente para a tomada de decisão do agente



Exemplo



Contraexemplo

Quiz

Quais das seguintes representações de estado satisfazem a propriedade de Markov?

(Alguns) Fundamentos

- **Markov decision process (MDP)**
  - Propriedade de Markov: o estado atual fornece toda informação relevante sobre o ambiente para a tomada de decisão do agente
- Assume-se:
  - único agente
  - sequencial (múltiplas decisões ao longo do tempo)
  - tempo discreto $t = 1, 2, \dots, T$ (uma ação e transição por timestep)
  - estacionário (estado muda, dinâmicas não)

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s
  - $\mathcal{P}$: função de **transição** de estados
    - $p(s', r \mid s, a) = \Pr\{S_t = s' \mid S_{t=1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$ é a probabilidade do agente receber a recompensa r e alcançar o estado s' após executar a ação a no estado s

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s
  - $\mathcal{P}$: função de **transição** de estados
    - $p(s', r \mid s, a) = \Pr\{S_t = s' \mid S_{t=1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$ é a probabilidade do agente receber a recompensa r e alcançar o estado s' após executar a ação a no estado s
  - $\mathcal{R}$: função de **recompensa**
    - $r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t=1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$ é a recompensa recebida após executar a ação a no estado s

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s
  - $\mathcal{P}$: função de **transição** de estados
    - $p(s', r \mid s, a) = \Pr\{S_t = s' \mid S_{t=1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$ é a probabilidade do agente receber a recompensa r e alcançar o estado s' após executar a ação a no estado s
  - $\mathcal{R}$: função de **recompensa**
    - $r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t=1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$ é a recompensa recebida após executar a ação a no estado s
  - $\gamma \in [0,1]$: fator de **desconto** sobre recompensas futuras

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s
  - $\mathcal{P}$: função de **transição** de estados
    - $p(s', r \mid s, a) = \Pr\{S_t = s' \mid S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$ é a probabilidade do agente receber a recompensa r e alcançar o estado s' após executar a ação a no estado s
  - $\mathcal{R}$: função de **recompensa**
    - $r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$ é a recompensa recebida após executar a ação a no estado s
  - $\gamma \in [0,1]$: fator de **desconto** sobre recompensas futuras

Representação flexível!

(Alguns) Fundamentos

- Tupla $(\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$:
  - $\mathcal{S}$: conjunto finito de **estados**
    - $S_t \in \mathcal{S}$ é o estado visitado no tempo t
  - $\mathcal{A}(s)$: conjunto finito de **ações** disponíveis no estado s
    - $A_t \in \mathcal{A}(s)$ é a ação executada no tempo t, quando no estado s
  - $\mathcal{P}$: função de **transição** de estados
    - $p(s', r \mid s, a) = \Pr\{S_t = s' \mid S_{t=1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a)$ é a probabilidade do agente receber a recompensa r e alcançar o estado s' após executar a ação a no estado s
  - $\mathcal{R}$: função de **recompensa**
    - $r(s, a) = \mathbb{E}[R_t \mid S_{t-1} = s, A_{t=1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r \mid s, a)$ é a recompensa recebida após executar a ação a no estado s
  - $\gamma \in [0,1]$: fator de **desconto** sobre recompensas futuras

Performance depende de modelagem adequada!

Princípio algorítmico

Abordagens

- **Programação Dinâmica**
  - Temos um modelo perfeito do ambiente na forma de um MDP
- **Monte Carlo**
  - NÃO temos um modelo perfeito
  - Podemos estimar funções de valor utilizando trajetórias completas
- **Diferença Temporal**
  - NÃO temos um modelo perfeito
  - Podemos estimar funções de valor utilizando transições entre estados

Abordagens

- **Programação Dinâmica**
  - Temos um modelo perfeito do ambiente na forma de um MDP
- **Monte Carlo**
  - NÃO temos um modelo perfeito
  - Podemos estimar funções de valor utilizando trajetórias completas
- **Diferença Temporal**
  - NÃO temos um modelo perfeito
  - Podemos estimar funções de valor utilizando transições entre estados

Diferença temporal

- *“If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning”*



Richard Sutton



Andrew Barto

Diferença temporal

- *“If one had to identify one idea as central and novel to reinforcement learning, it would undoubtedly be temporal-difference (TD) learning”*
- Agente deve aprender uma política **interagindo** com o ambiente



Richard Sutton



Andrew Barto

Quiz

Como um agente pode coletar experiências para aprender a tomar melhores decisões?

Exploração vs. aproveitamento

- Agente deve:
  - **Explorar** ambiente (para entender sua dinâmica)
  - **Aproveitar** conhecimento (para ter boas recompensas)

Exploração vs. aproveitamento

- Agente deve:
  - **Explorar** ambiente (para entender sua dinâmica)
  - **Aproveitar** conhecimento (para ter boas recompensas)
- Como balancear exploração e aproveitamento?

Exploração vs. aproveitamento

- Como balancear exploração e aproveitamento?
  - ***ε*-greedy**: explora com probabilidade ε, aproveita com probabilidade $1 - \varepsilon$
  - **Soft-max**: seleciona ações com probabilidade proporcional ao seu valor

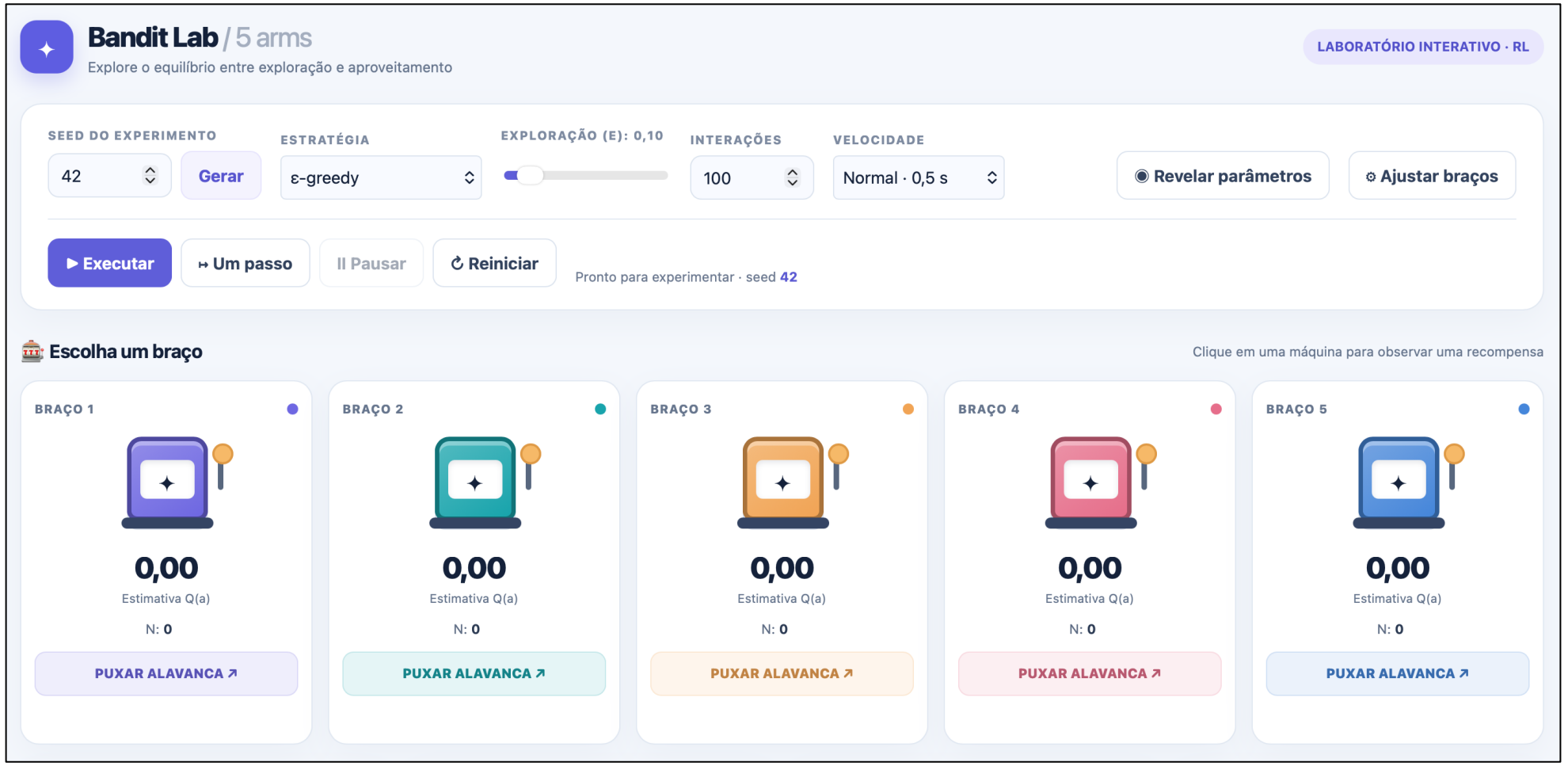
$$\Pr\{A_t = a\} \doteq \frac{e^{H_t(a)}}{\sum_{b=1}^{k} e^{H_t(b)}} \doteq \pi_t(a)$$

  - **Upper-confidence bounds**: age de forma otimista diante da incerteza

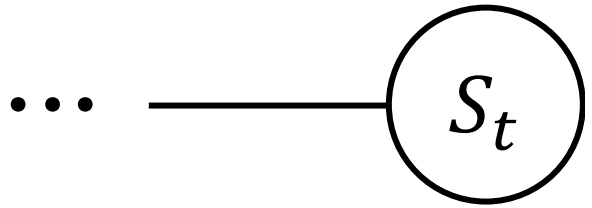
$$A_t \doteq \arg\max_a \left[Q_t(a) + c\sqrt{\frac{\ln t}{N_t(a)}} \right]$$

  - Gittins, Thompson sampling, etc.

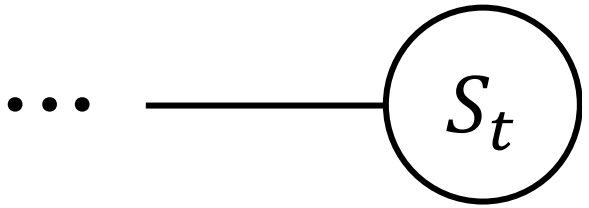
Exploração vs. aproveitamento



Controle baseado em diferença temporal

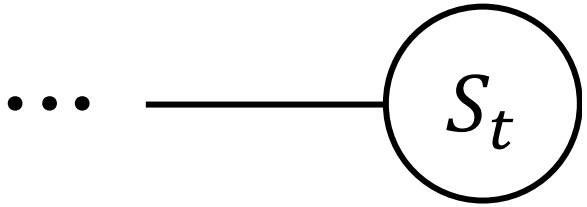


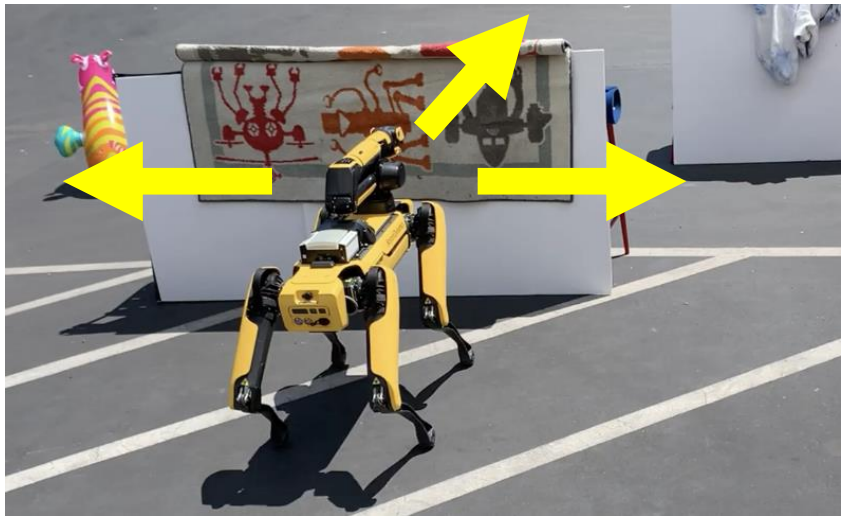
Controle baseado em diferença temporal



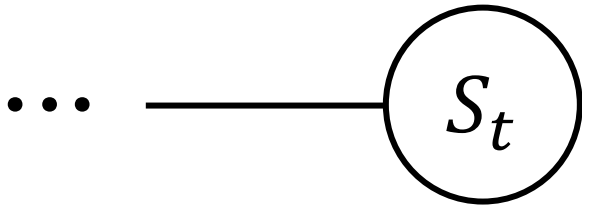


Controle baseado em diferença temporal



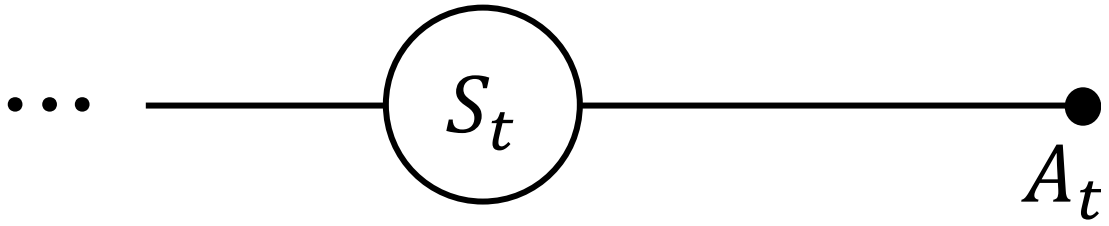


Controle baseado em diferença temporal



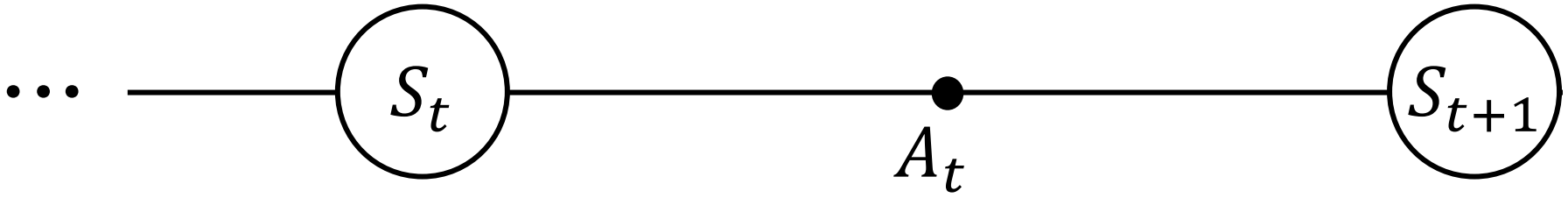


Controle baseado em diferença temporal



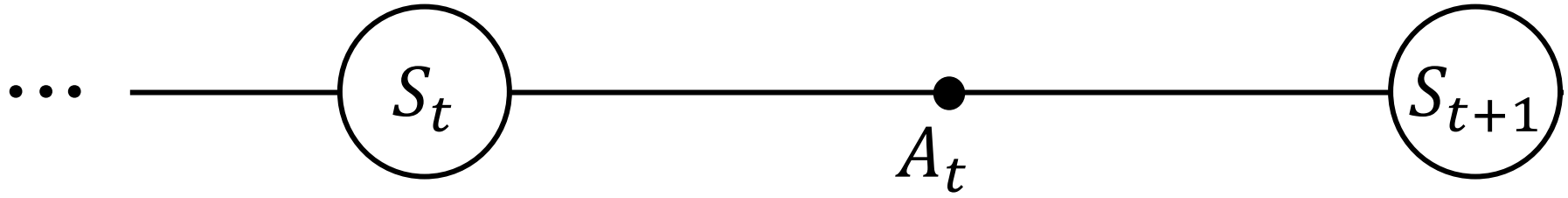


Controle baseado em diferença temporal

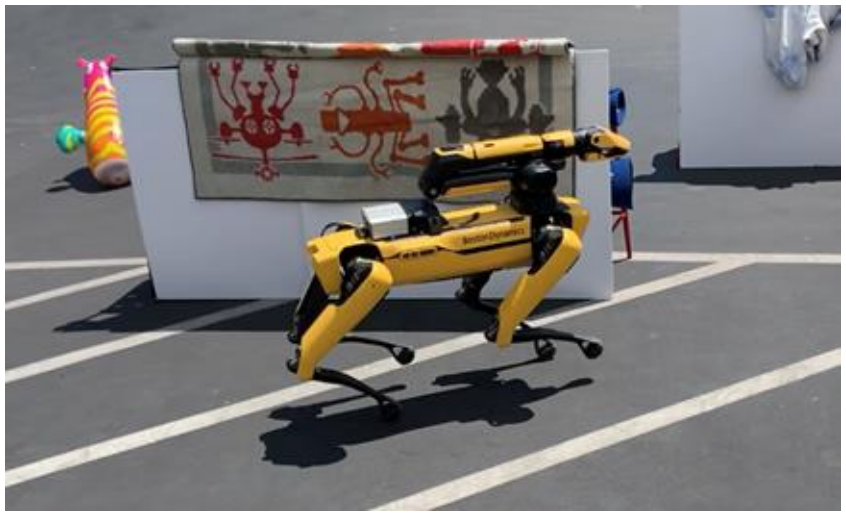




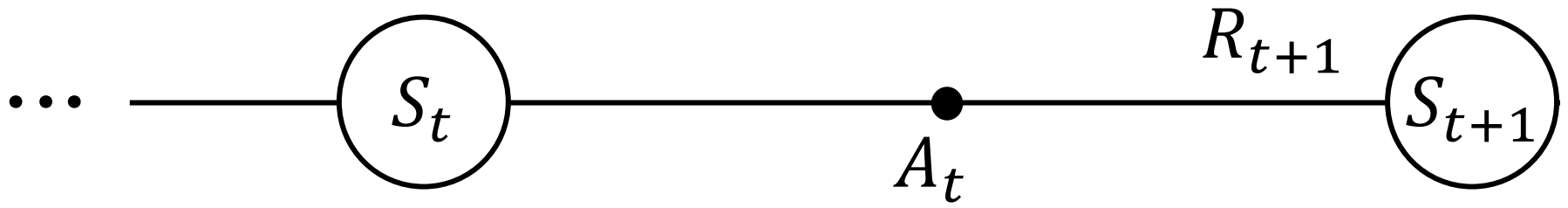
Controle baseado em diferença temporal



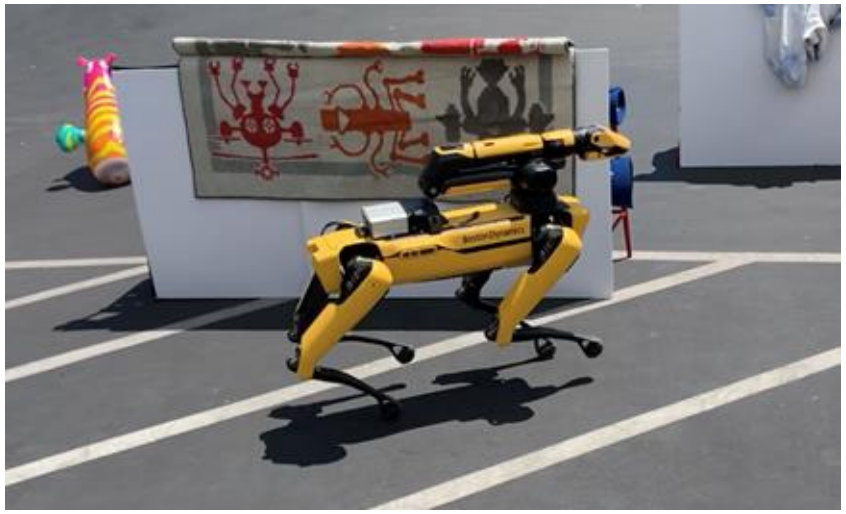




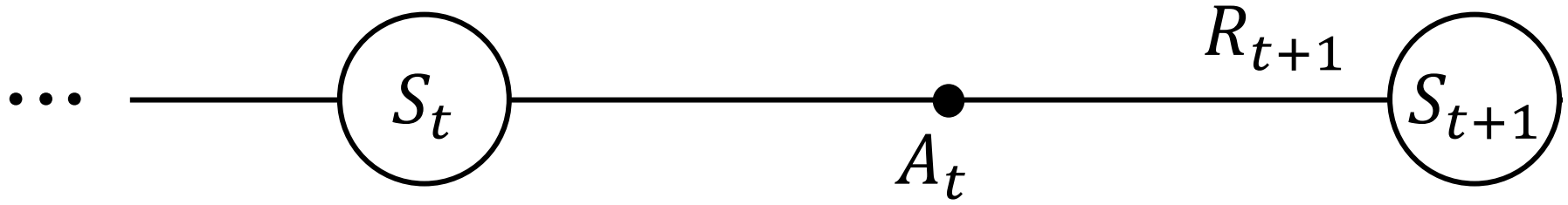
Controle baseado em diferença temporal







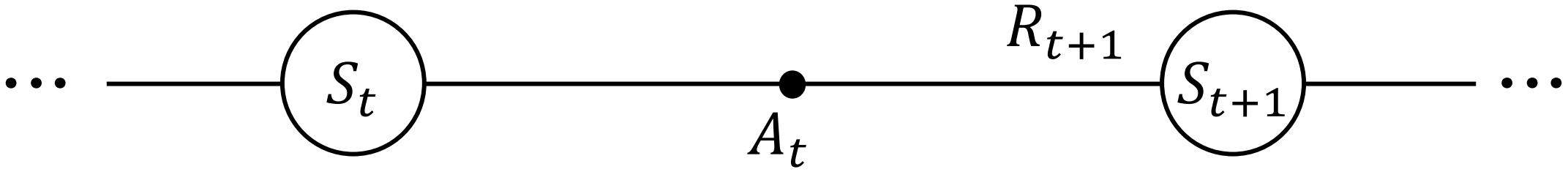
Controle baseado em diferença temporal







Controle baseado em diferença temporal

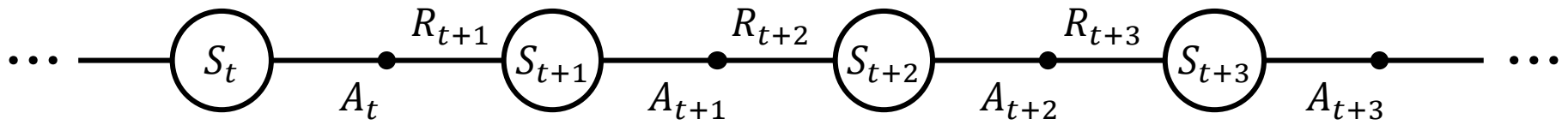






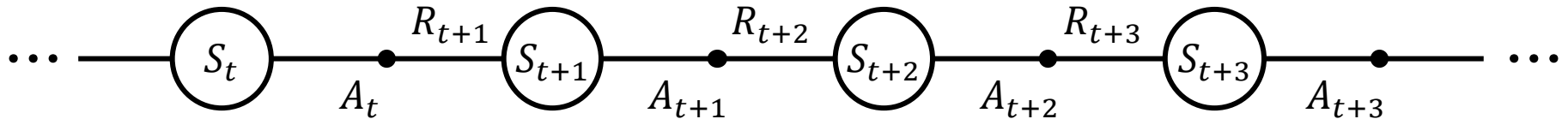
Controle baseado em diferença temporal

- Episódio:



Controle baseado em diferença temporal

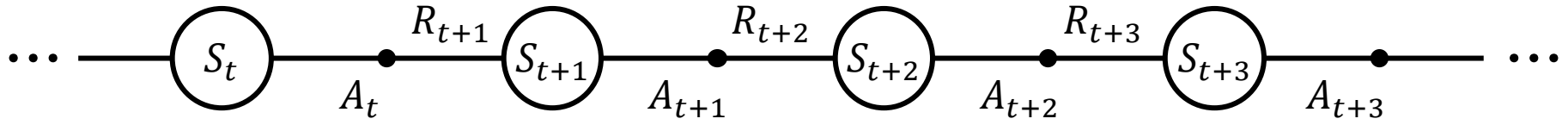
- Episódio:



- **Objetivo**: aprender uma política $\pi \approx \pi^*$

Controle baseado em diferença temporal

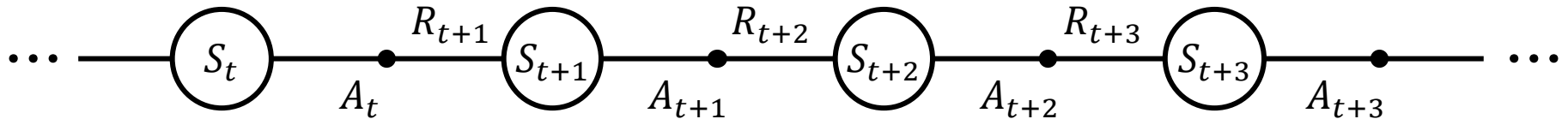
- Episódio:



- **Objetivo**: aprender uma política $\pi \approx \pi^*$
- **Experiências** coletadas permitem **aproximar** as funções de valor

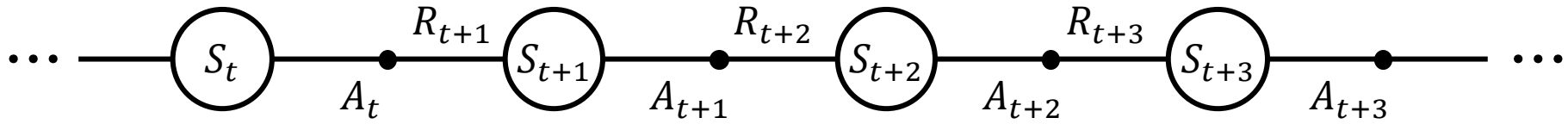
Controle baseado em diferença temporal

- Episódio:

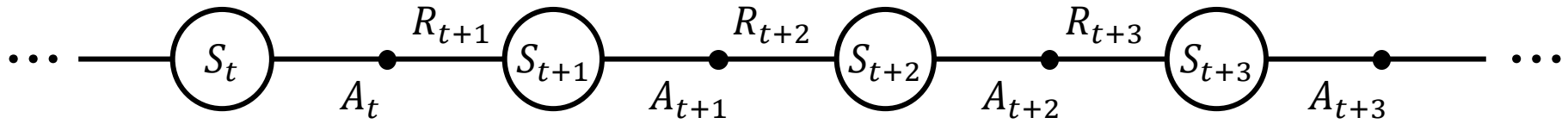


- **Objetivo**: aprender uma política $\pi \approx \pi^*$
- **Experiências** coletadas permitem **aproximar** as funções de valor
- Se a estimativa for adequada, é possível derivar a **política ótima**

SARSA

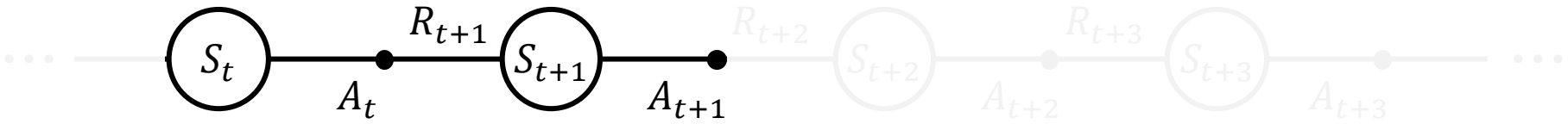


SARSA



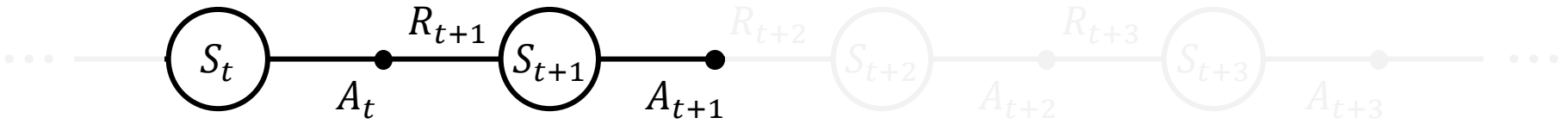
- Aprende uma função de valor de ação Q que aproxima Q^*

SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

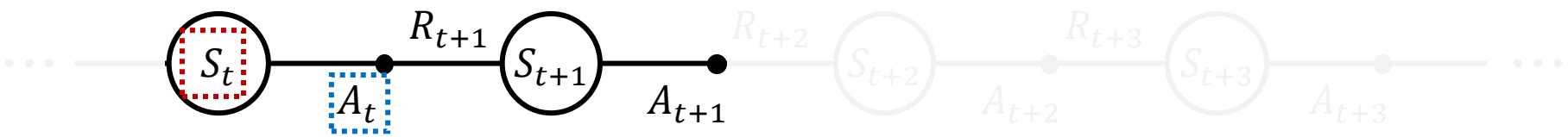
SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

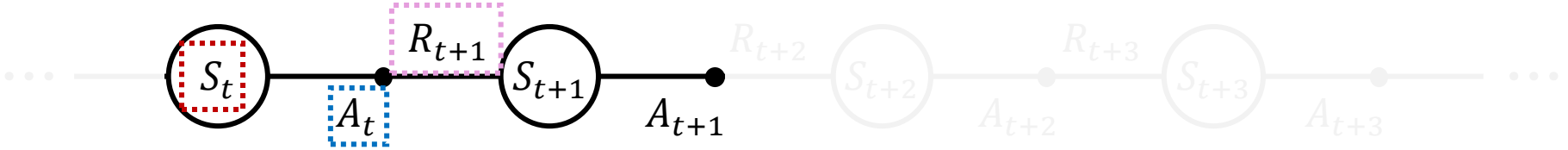
SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

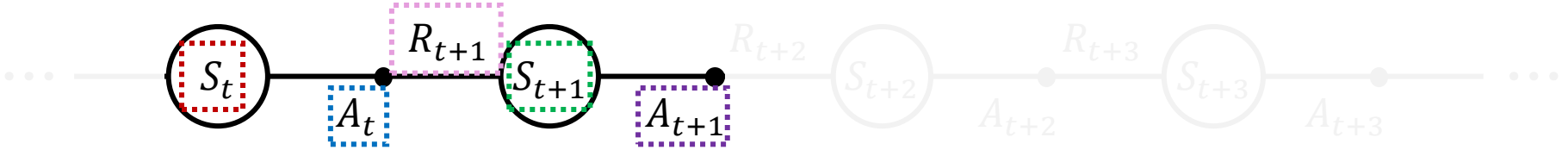
SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

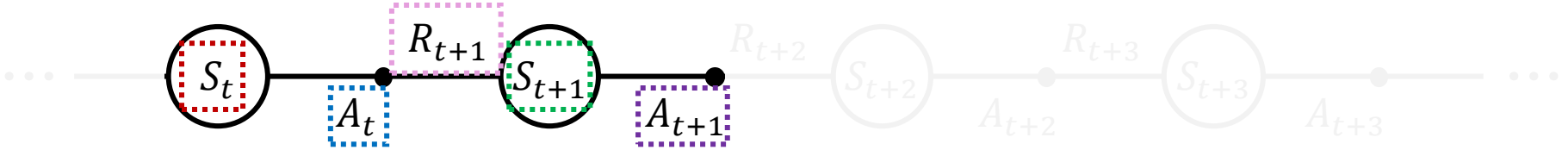
SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

SARSA



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- **Ideia**:
  - Começar com Qs aleatórios
  - Atualizá-los repetidamente conforme o agente interage com o ambiente
  - Até encontrar Q^*

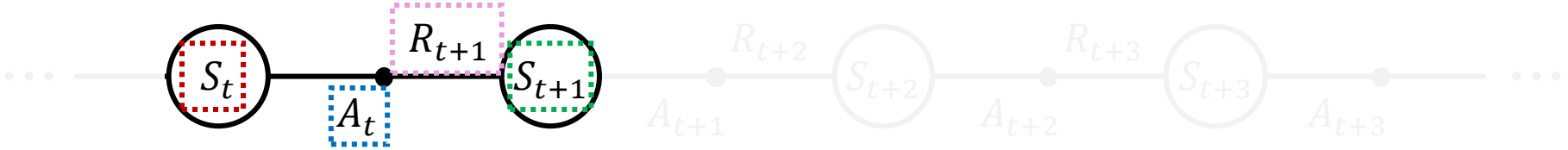
SARSA

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
    Initialize S
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Loop for each step of episode:
        Take action A, observe R, S'
        Choose A' from S' using policy derived from Q (e.g., ε-greedy)
        $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
        $S \leftarrow S'$; $A \leftarrow A'$;
    until S is terminal

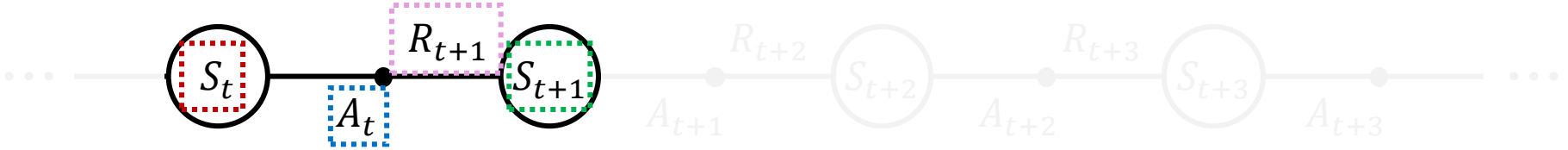
Q-Learning



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

Q-Learning



- Aprende uma função de valor de ação Q que aproxima Q^*

$$Q(S_t, A_t) = Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

- Assume escolha da **melhor ação conhecida** no estado seguinte
- Princípio de funcionamento parecido com SARSA

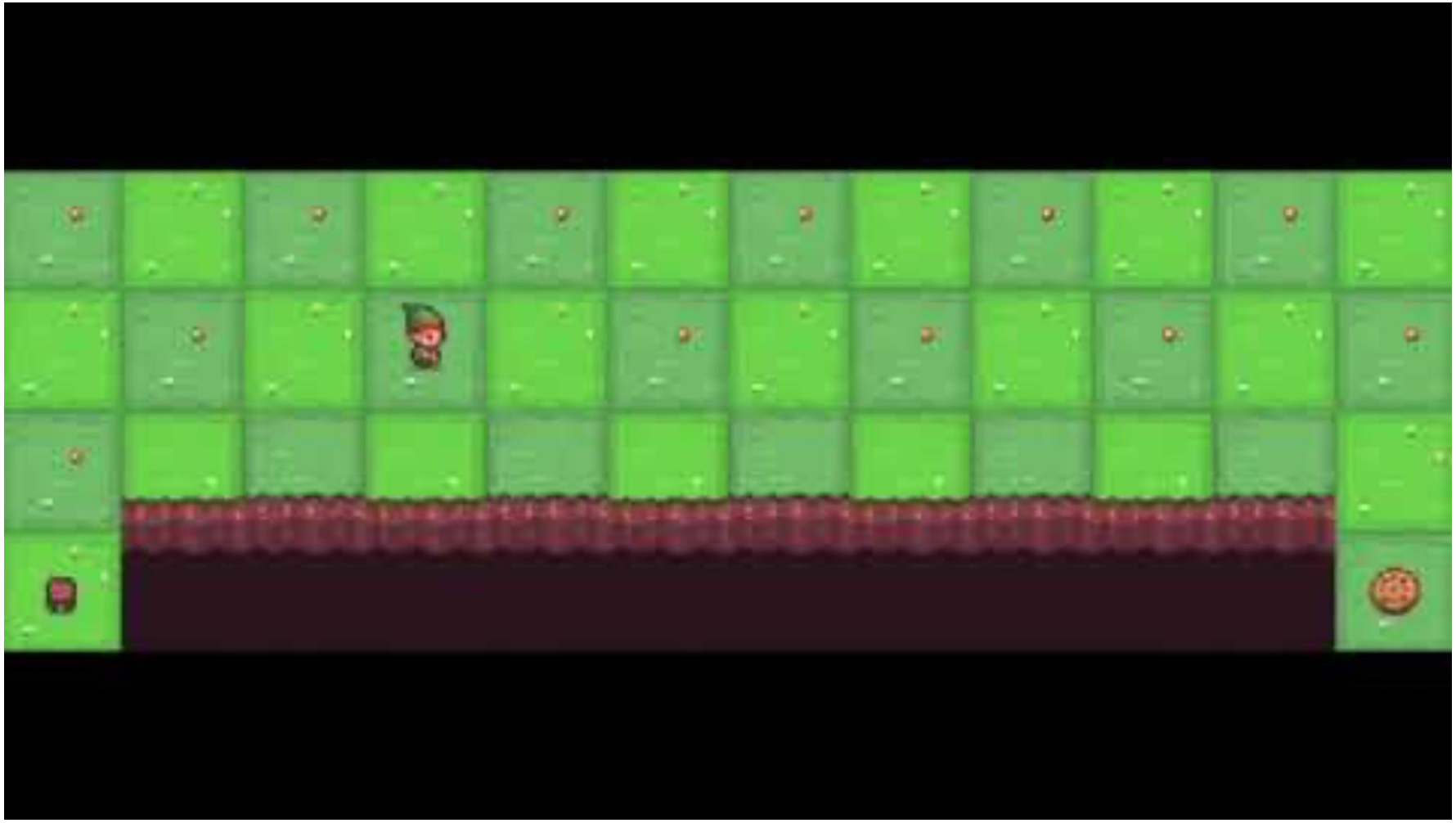
Q-Learning

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
    Initialize S
    Loop for each step of episode:
        Choose A from S using policy derived from Q (e.g., ε-greedy)
        Take action A, observe R, S'
        $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
        $S \leftarrow S'$
    until S is terminal

Q-Learning



https://youtu.be/xGmyV6RE33U

Prática

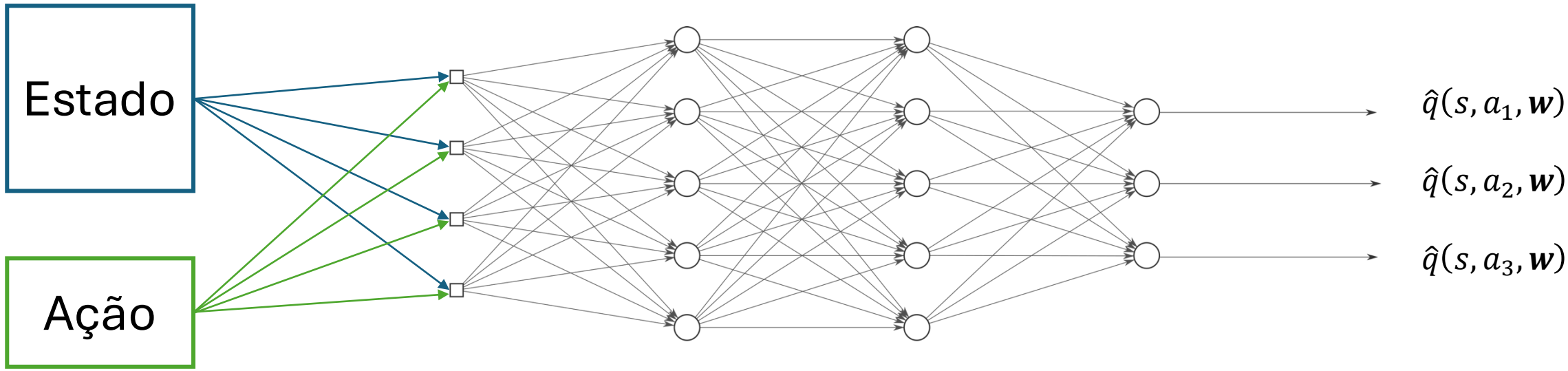
Acesse aqui o notebook no Colab



Controle com aproximação de função

- Devemos aproximar a função de valor de ação

$$\hat{q}(s, a, \boldsymbol{w}) \approx q_\pi(s, a)$$



Deep Q-Network (DQN)

- Combina Q-Learning com redes neurais profundas
- **Experience replay**: armazena tuplas de experiência para posterior treino da rede neural
- Par de redes para melhorar a **estabilidade**

Deep Q-Network (DQN)

- Atualização dos pesos:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[R_{t+1} + \gamma \max_a \tilde{q}(S_{t+1}, a, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \Big] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

Deep Q-Network (DQN)

- Atualização dos pesos:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \Big[R_{t+1} + \gamma \max_a \tilde{q}(S_{t+1}, a, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \Big] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

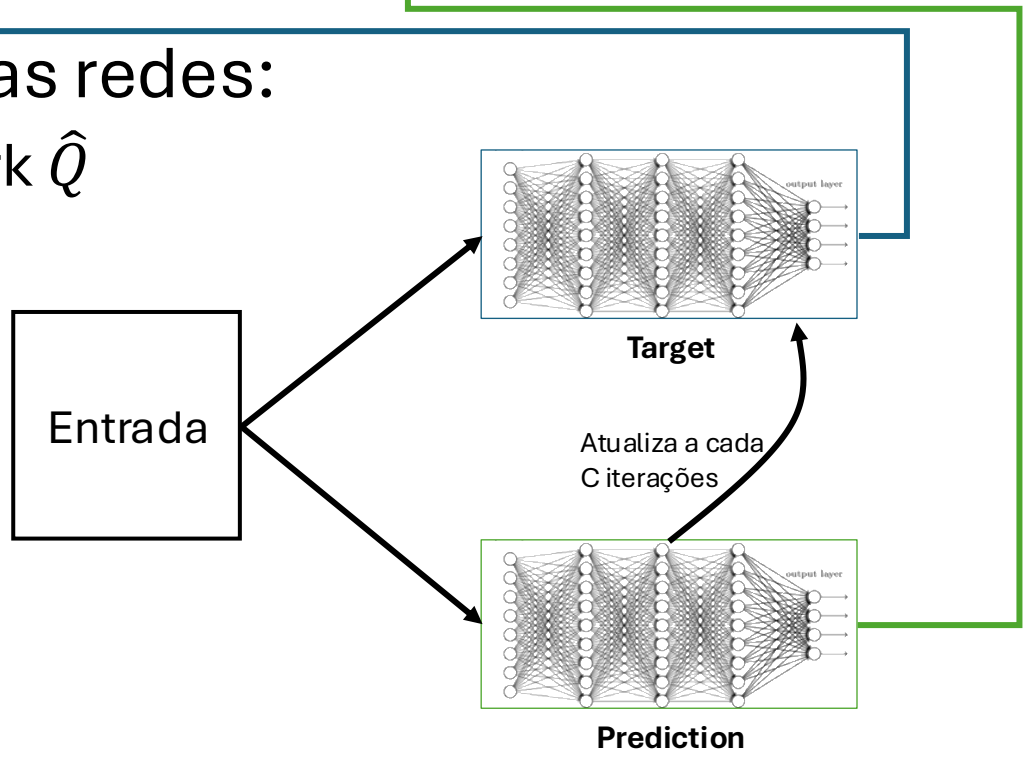
- Estabilidade melhorada com duas redes:
  - (i) Prediction (action-value) network $\hat{Q}$
  - (ii) Target net $\tilde{Q}$

Deep Q-Network (DQN)

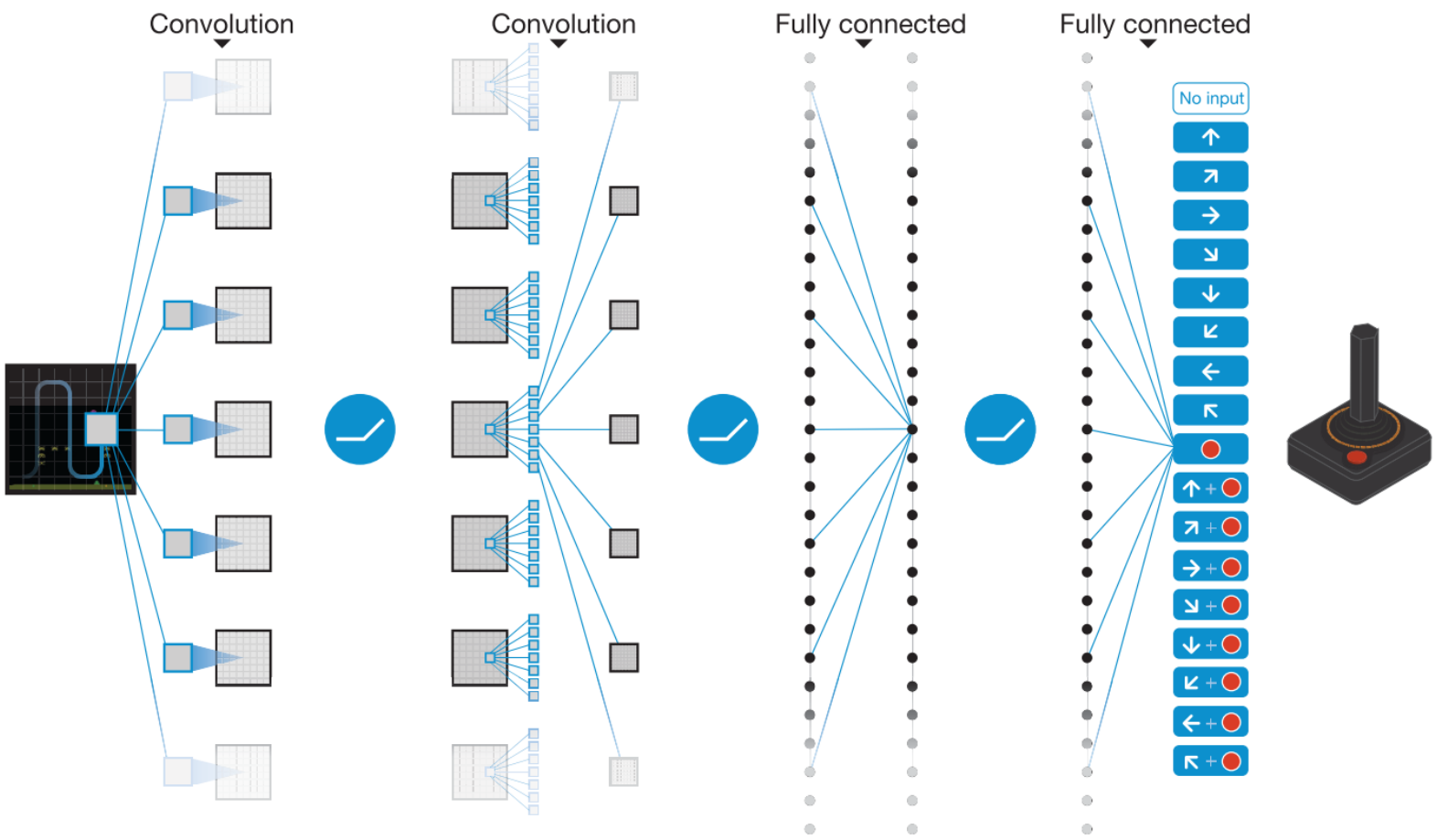
- Atualização dos pesos:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha \left[R_{t+1} + \gamma \max_a \tilde{q}(S_{t+1}, a, \mathbf{w}_t) - \hat{q}(S_t, A_t, \mathbf{w}_t) \right] \nabla \hat{q}(S_t, A_t, \mathbf{w}_t)$$

- Estabilidade melhorada com duas redes:
  - (i) Prediction (action-value) network $\hat{Q}$
  - (ii) Target net $\tilde{Q}$



Deep Q-Network (DQN)

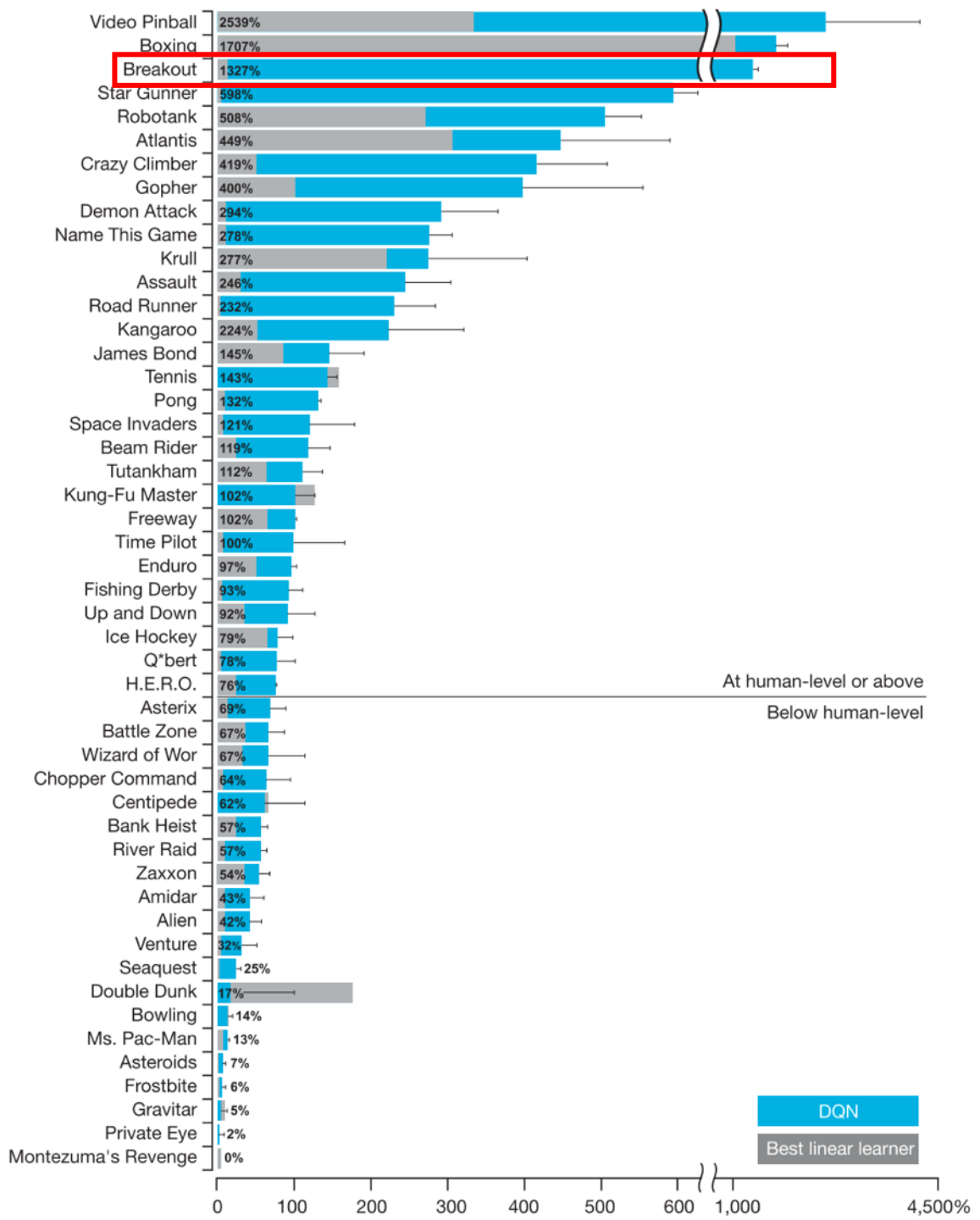


Deep Q-Network (DQN)

- Atari 2600 benchmark

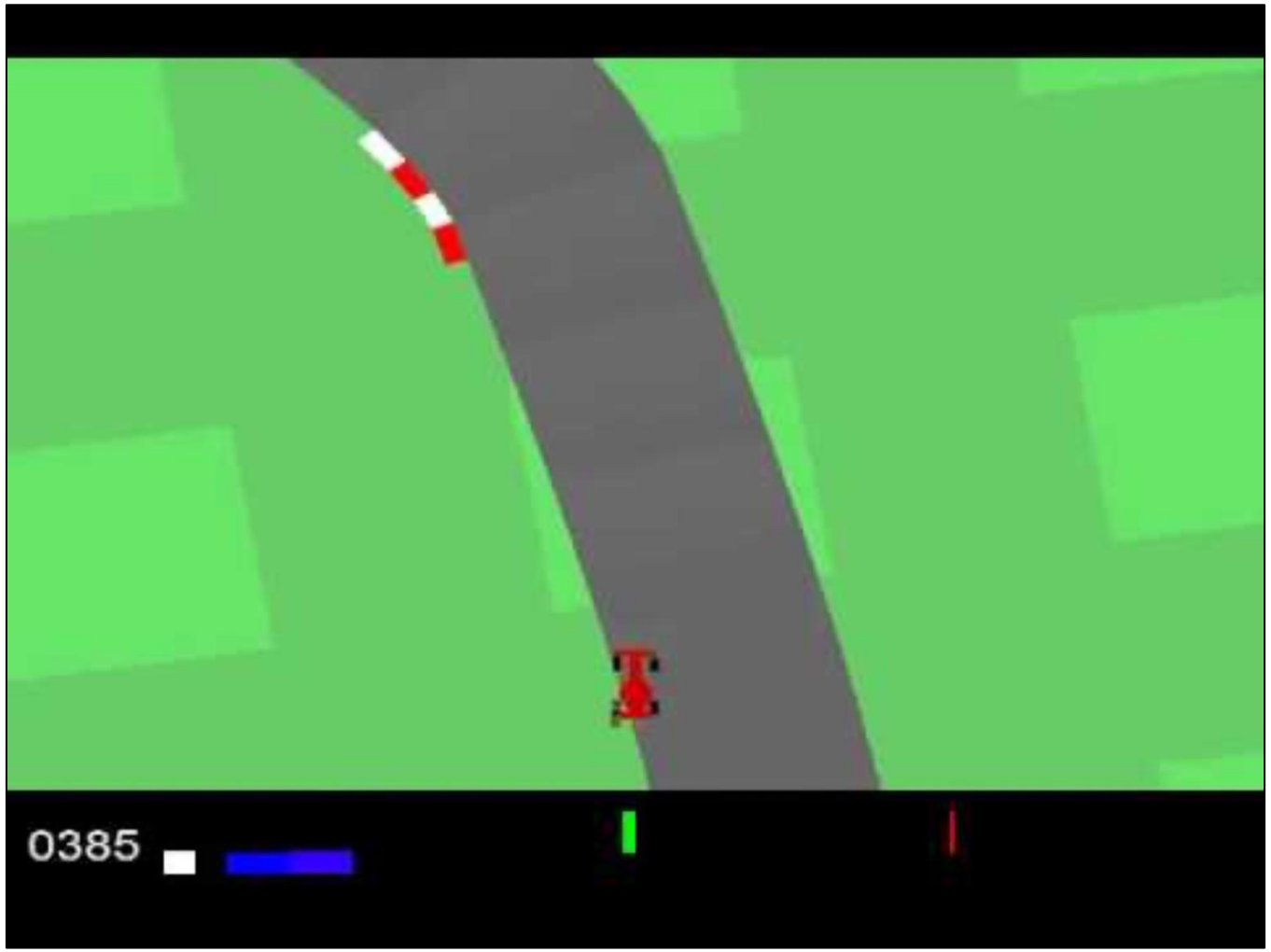


https://www.youtube.com/watch?v=V1eYniJ0Rnk



Aplicações

Aplicações



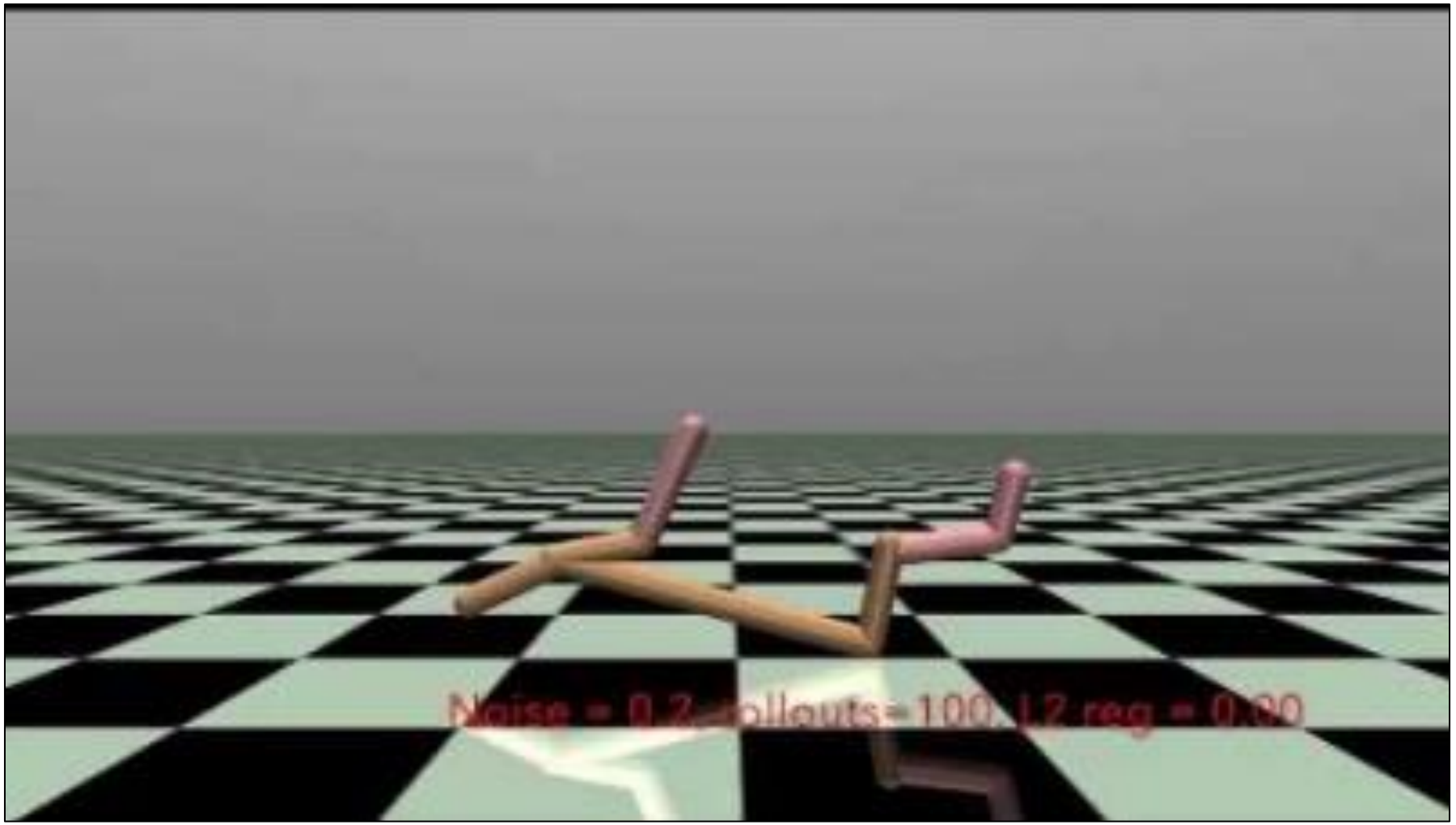
https://www.youtube.com/watch?v=pTYOyl8To7g

Aplicações



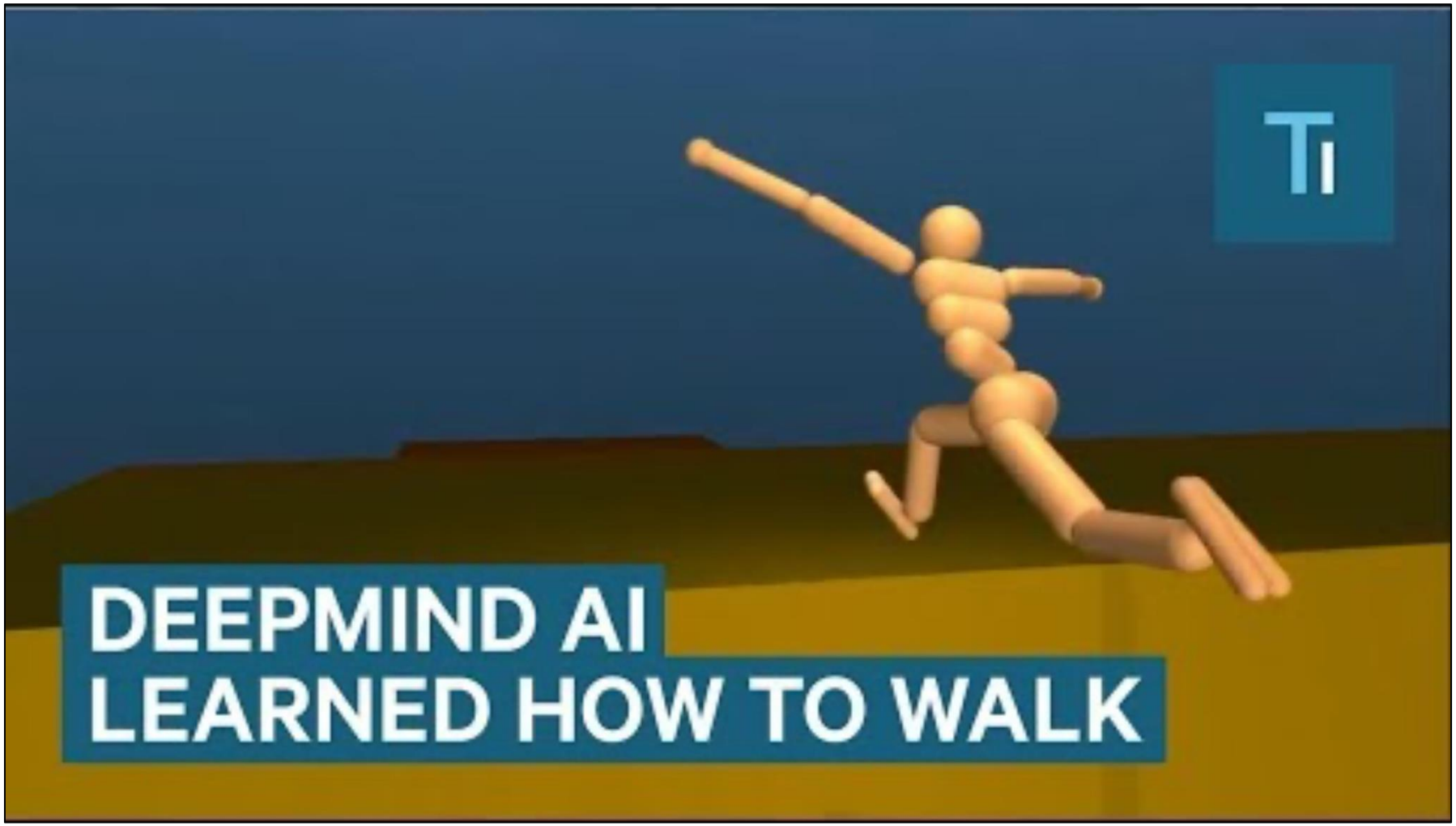
https://www.youtube.com/watch?v=L4KBBAwF_bE

Aplicações



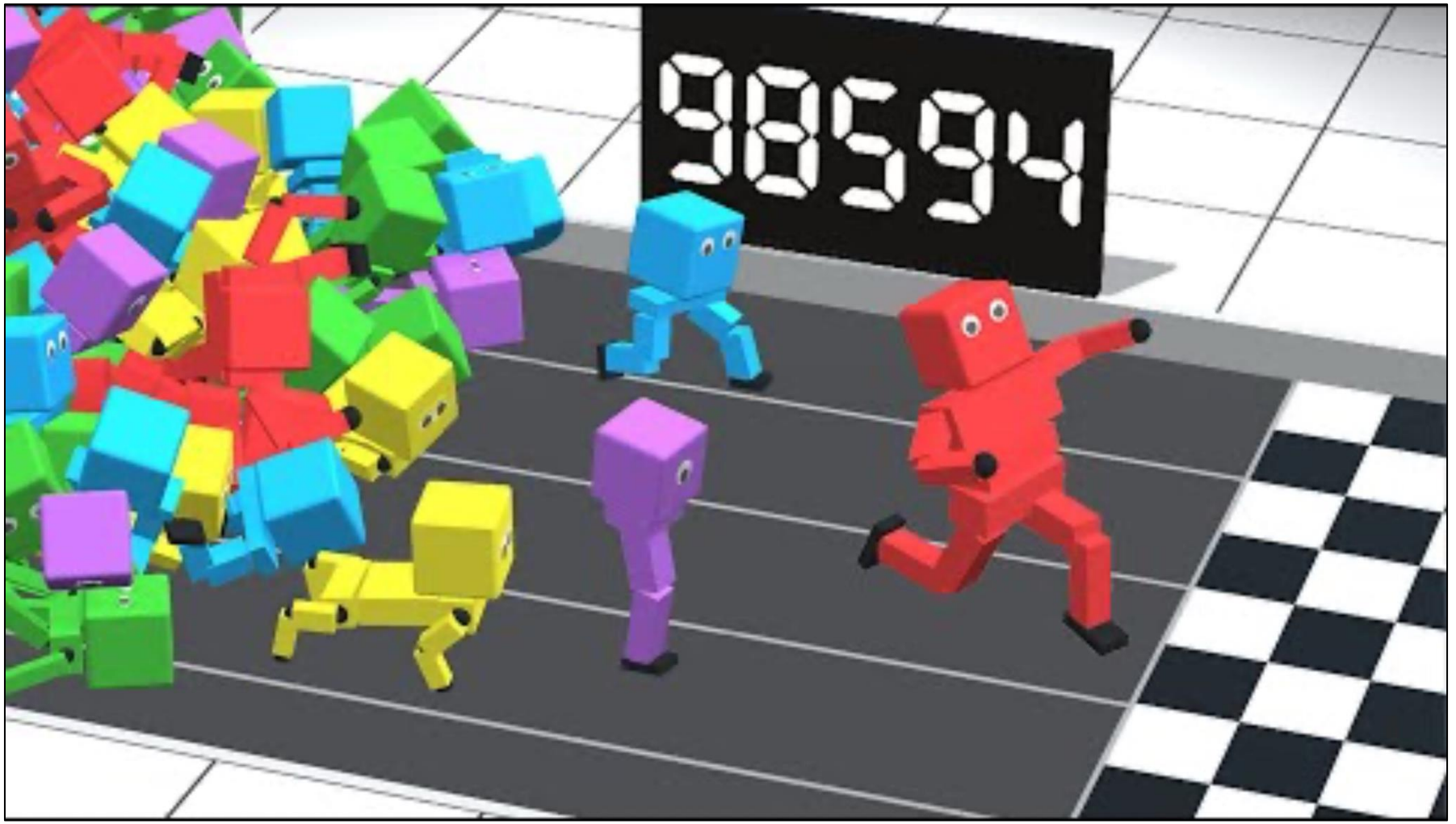
https://www.youtube.com/watch?v=qU8Nd9lyxlw

Aplicações



https://www.youtube.com/watch?v=gn4nRCC9TwQ

Aplicações



https://www.youtube.com/watch?v=pJPdW8WWAso

Aplicações



https://www.youtube.com/watch?v=kopoLzvh5jY

Ah, mas RL só serve para controlar joguinhos



AlphaGo



https://www.youtube.com/watch?v=vFr3K2DORc8

AlphaStar



https://www.youtube.com/watch?v=6EQAsrfUIyo

AlphaChip

Article

A graph placement methodology for fast chip design

https://doi.org/10.1038/s41586-021-03544-w

Received: 3 November 2020

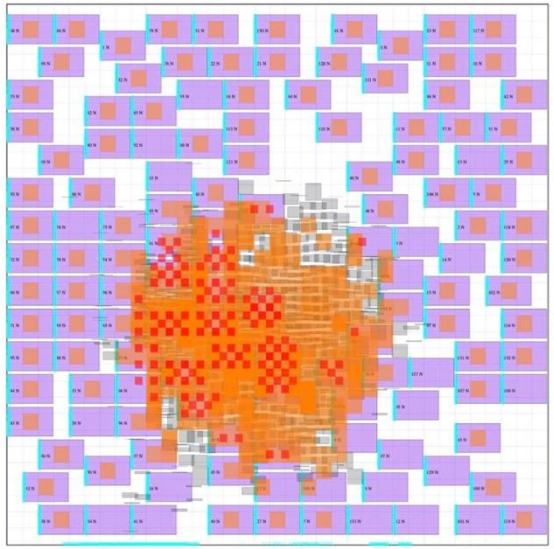
Accepted: 13 April 2021

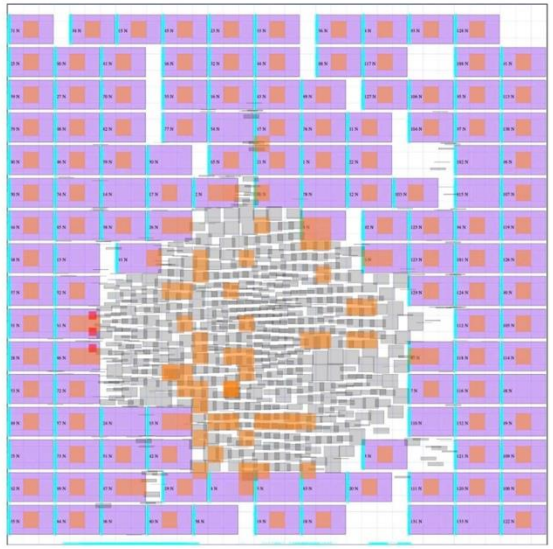
Published online: 9 June 2021

Check for updates

Azalia Mirhoseini[1,4]✉, Anna Goldie[1,3,4]✉, Mustafa Yazgan[2], Joe Wenjie Jiang[1], Ebrahim Songhori[1], Shen Wang[1], Young-Joon Lee[2], Eric Johnson[1], Omkar Pathak[2], Azade Nazi[1], Jiwoo Pak[2], Andy Tong[2], Kavya Srinivasa[2], William Hang[3], Emre Tuncer[2], Quoc V. Le[1], James Laudon[1], Richard Ho[2], Roger Carpenter[2] & Jeff Dean[1]

Chip floorplanning is the engineering task of designing the physical layout of a computer chip. Despite five decades of research[1], chip floorplanning has defied automation, requiring months of intense effort by physical design engineers to produce manufacturable layouts. Here we present a deep reinforcement learning approach to chip floorplanning. In under six hours, our method automatically generates chip floorplans that are superior or comparable to those produced by humans in all key metrics, including power consumption, performance and chip area. To achieve this, we pose chip floorplanning as a reinforcement learning problem, and develop an edge-based graph convolutional neural network architecture capable of learning rich and transferable representations of the chip. As a result, our method utilizes past experience to become better and faster at solving new instances of the problem, allowing chip design to be performed by artificial agents with more





Google Loon



https://youtu.be/PBdGge2ipCg?t=2014

Controle de reatores nucleares

Article

Magnetic control of tokamak plasmas through deep reinforcement learning

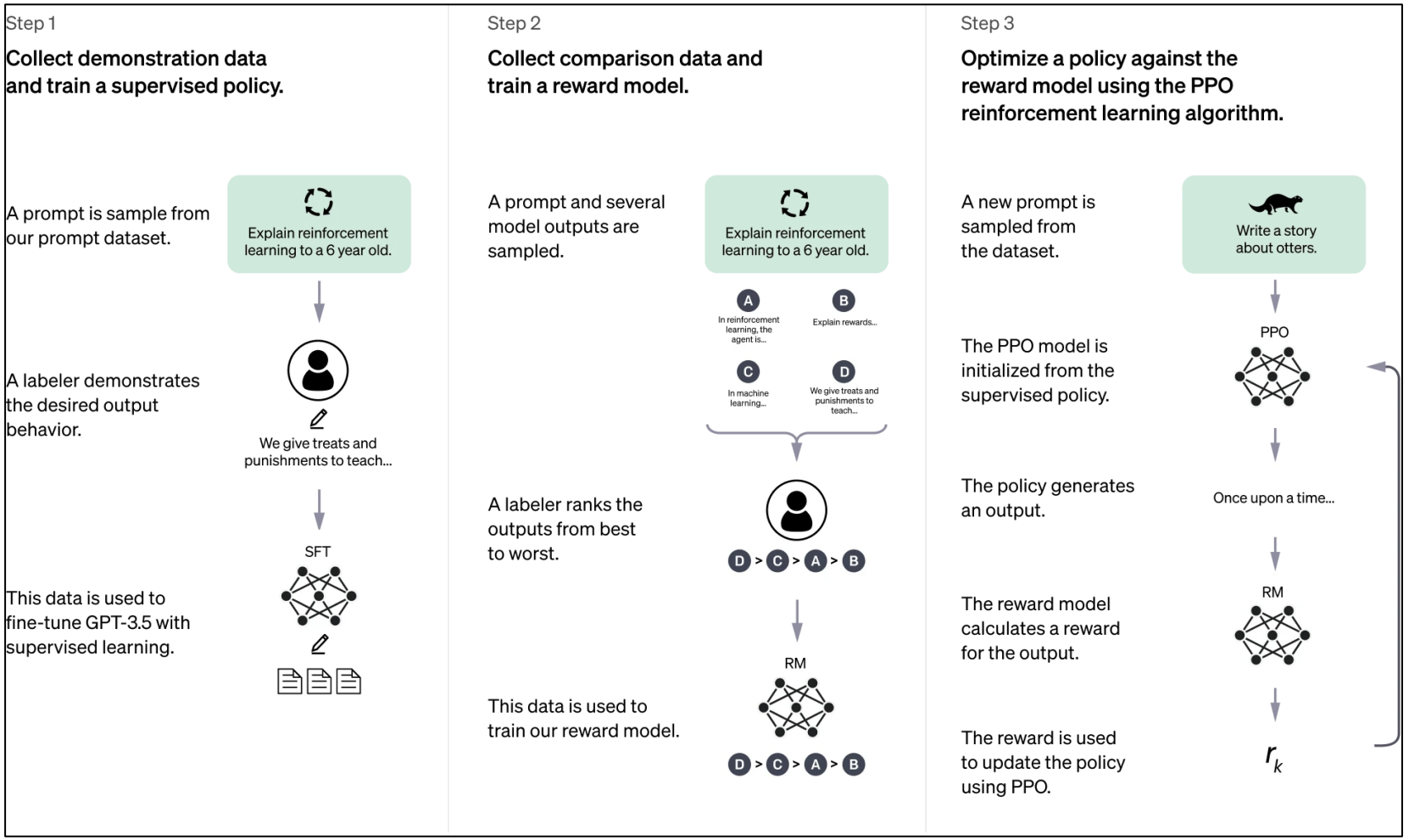
https://doi.org/10.1038/s41586-021-04301-9
Received: 14 July 2021
Accepted: 1 December 2021
Published online: 16 February 2022
Open access
Check for updates

Jonas Degrave[1,3], Federico Felici[2,3]✉, Jonas Buchli[1,3]✉, Michael Neunert[1,3], Brendan Tracey[1,3]✉, Francesco Carpanese[1,2,3], Timo Ewalds[1,3], Roland Hafner[1,3], Abbas Abdolmaleki[1], Diego de las Casas[1], Craig Donner[1], Leslie Fritz[1], Cristian Galperti[2], Andrea Huber[1], James Keeling[1], Maria Tsimpoukelli[1], Jackie Kay[1], Antoine Merle[2], Jean-Marc Moret[2], Seb Noury[1], Federico Pesamosca[2], David Pfau[1], Olivier Sauter[2], Cristian Sommariva[2], Stefano Coda[2], Basil Duval[2], Ambrogio Fasoli[2], Pushmeet Kohli[1], Koray Kavukcuoglu[1], Demis Hassabis[1] & Martin Riedmiller[1,3]

Nuclear fusion using magnetic confinement, in particular in the tokamak configuration, is a promising path towards sustainable energy. A core challenge is to shape and maintain a high-temperature plasma within the tokamak vessel. This requires high-dimensional, high-frequency, closed-loop control using magnetic actuator coils, further complicated by the diverse requirements across a wide range of plasma configurations. In this work, we introduce a previously undescribed architecture for tokamak magnetic controller design that autonomously learns to command the full set of control coils. This architecture meets control objectives specified at a high level, at the same time satisfying physical and operational



ChatGPT



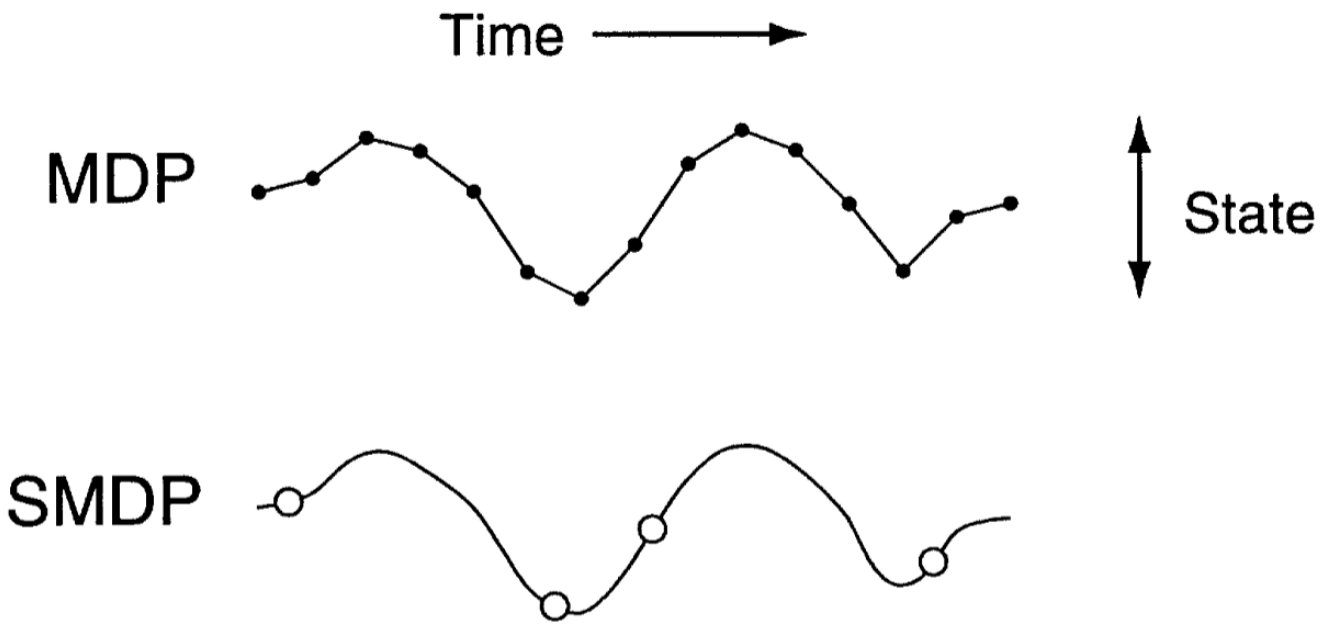
https://openai.com/index/chatgpt/

Options

Abstração temporal

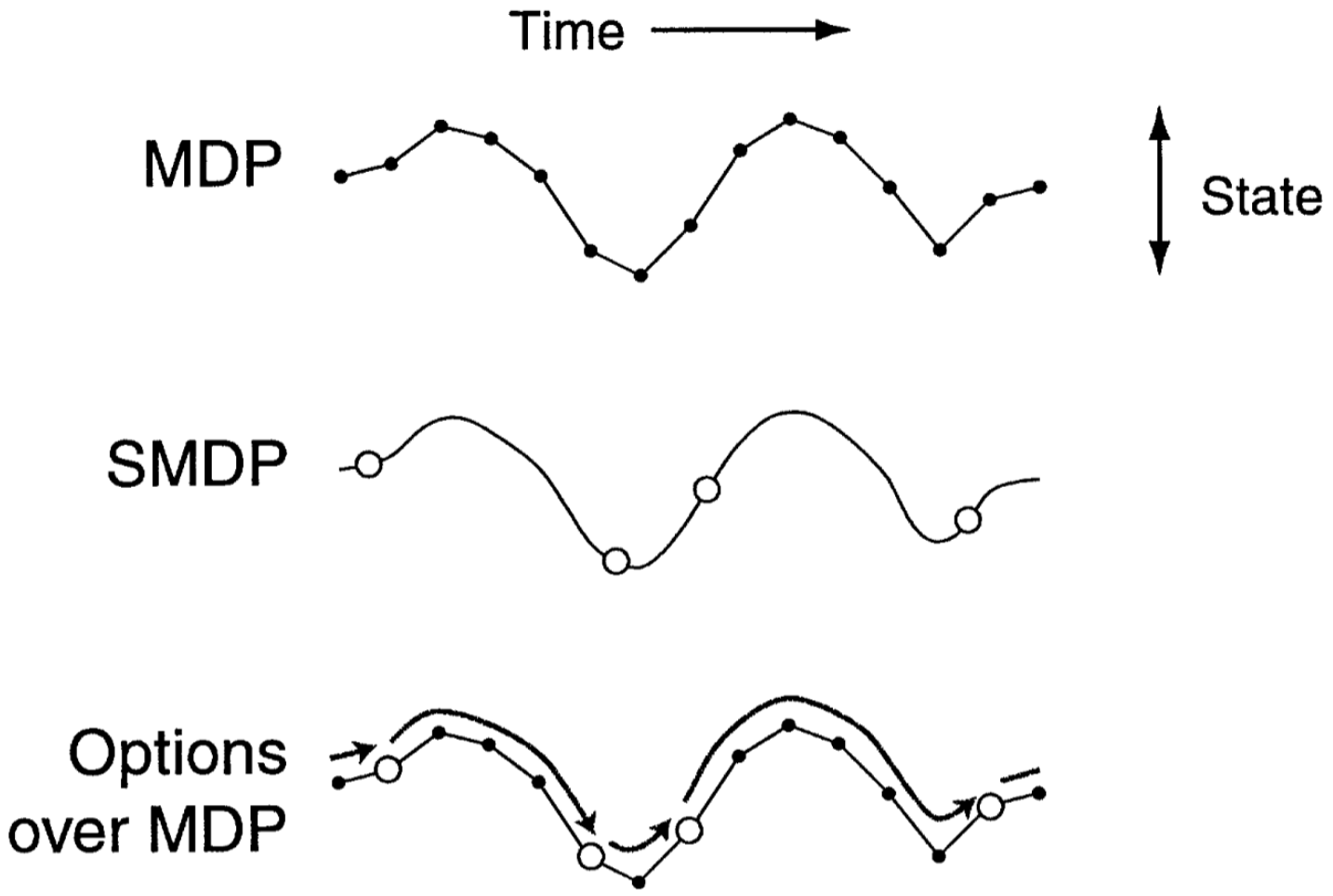


Abstração temporal



SUTTON, Richard S.; PRECUP, Doina; SINGH, Satinder. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, [s. l.], v. 112, n. 1–2, p. 181–211, 1999.

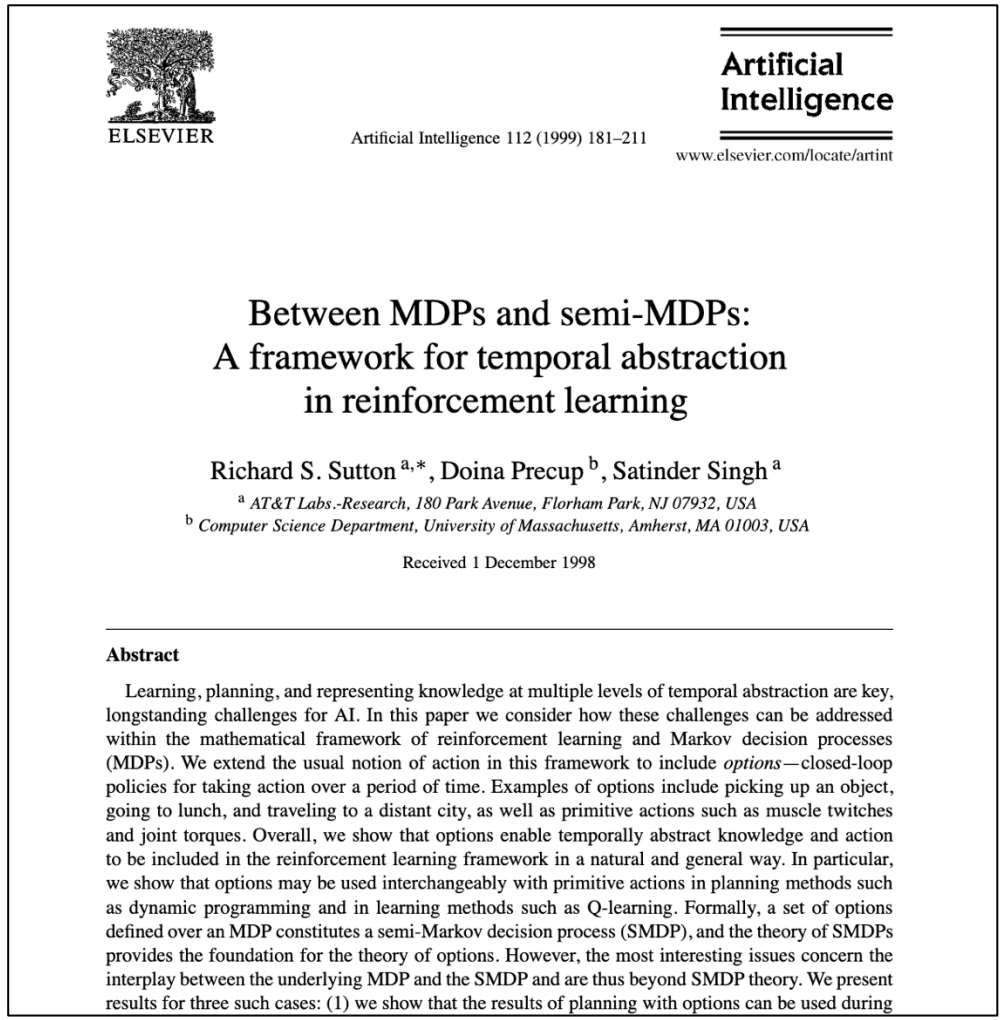
Abstração temporal



SUTTON, Richard S.; PRECUP, Doina; SINGH, Satinder. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. Artificial Intelligence, [s. l.], v. 112, n. 1–2, p. 181–211, 1999.

Options

- Comportamento de **alto nível** que pode ser escolhido pelo agente e que controla suas ações ao longo de **múltiplo passos**
- **Vantagens**: exploração, reutilização e planejamento



ELSEVIER

Artificial Intelligence 112 (1999) 181–211

Artificial Intelligence

www.elsevier.com/locate/artint

Between MDPs and semi-MDPs:
A framework for temporal abstraction
in reinforcement learning

Richard S. Sutton [a,*], Doina Precup [b], Satinder Singh [a]

[a] *AT&T Labs.-Research, 180 Park Avenue, Florham Park, NJ 07932, USA*

[b] *Computer Science Department, University of Massachusetts, Amherst, MA 01003, USA*

Received 1 December 1998

Abstract

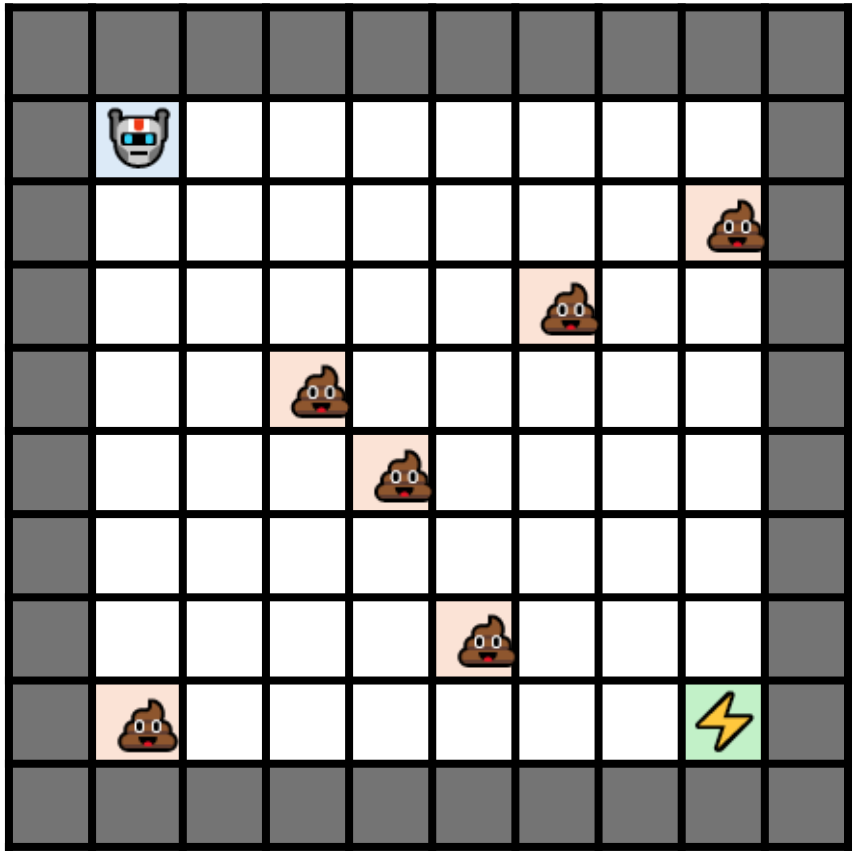
Learning, planning, and representing knowledge at multiple levels of temporal abstraction are key, longstanding challenges for AI. In this paper we consider how these challenges can be addressed within the mathematical framework of reinforcement learning and Markov decision processes (MDPs). We extend the usual notion of action in this framework to include *options*—closed-loop policies for taking action over a period of time. Examples of options include picking up an object, going to lunch, and traveling to a distant city, as well as primitive actions such as muscle twitches and joint torques. Overall, we show that options enable temporally abstract knowledge and action to be included in the reinforcement learning framework in a natural and general way. In particular, we show that options may be used interchangeably with primitive actions in planning methods such as dynamic programming and in learning methods such as Q-learning. Formally, a set of options defined over an MDP constitutes a semi-Markov decision process (SMDP), and the theory of SMDPs provides the foundation for the theory of options. However, the most interesting issues concern the interplay between the underlying MDP and the SMDP and are thus beyond SMDP theory. We present results for three such cases: (1) we show that the results of planning with options can be used during

Options

- Uma option $\omega \in \Omega$ é uma tupla $\omega = (I_\omega, \pi_\omega, \beta_\omega)$:
  - $I_\omega(s)$: estados nos quais a option pode ser iniciada
  - $\pi_\omega(s \mid a)$: política interna sobre ações primitivas
  - $\beta_\omega(s)$: probabilidade de encerrar a option em cada estado
- Uma option pode durar diversos passos
- Política sobre options $\pi_\Omega(\omega \mid s)$ seleciona uma option disponível no estado s

Options

- Option ω_1:
  - Começa quando bateria > 15%
  - Anda em zigue-zague aspirando o lixo
  - Termina quando bateria cai para 15% ou quando termina de percorrer ambiente
- Option ω_2:
  - Começa quando bateria <= 15%
  - Vai até estação de recarga e espera completar bateria
  - Termina quando bateria atinge 100%



Quiz

Quais afirmações estão corretas sobre as options do robô aspirador?

SMDP Q-learning

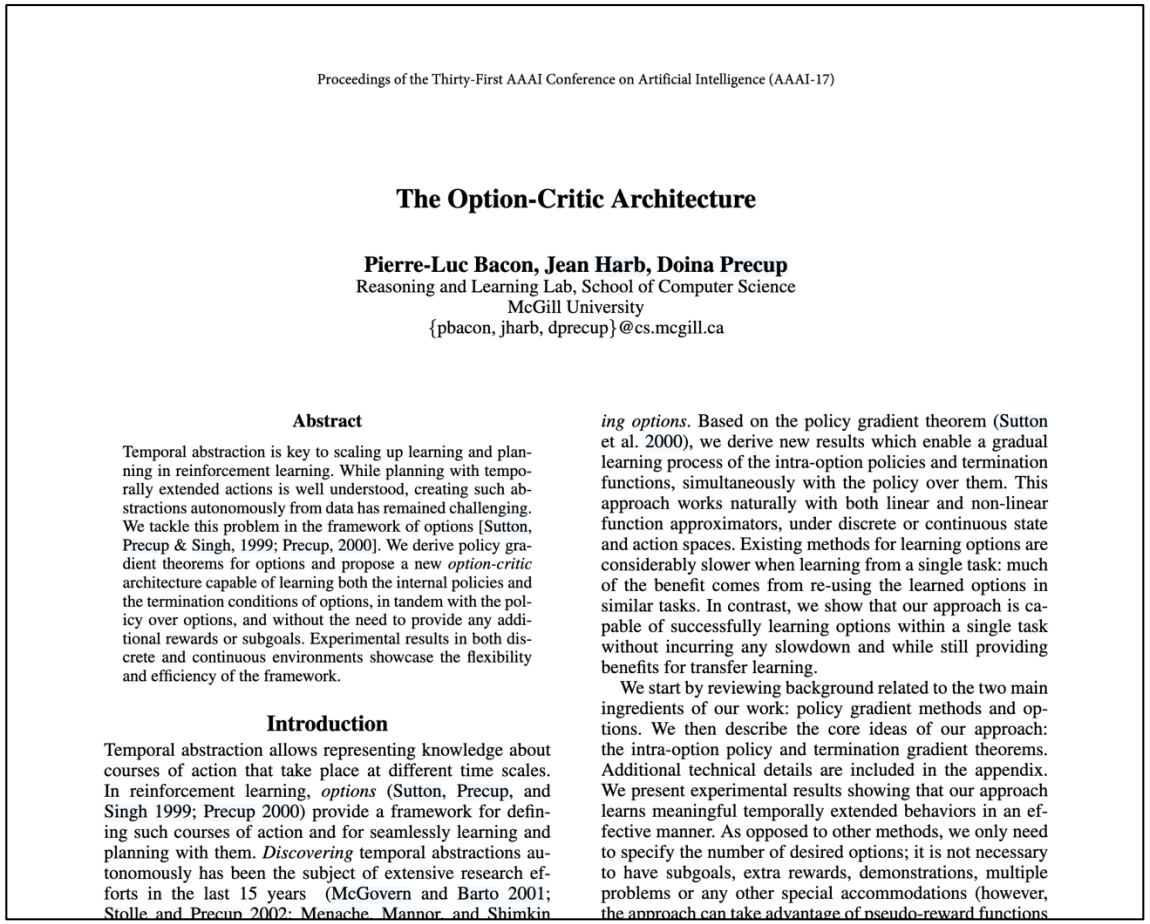
- Atualiza Q ao término da option:

$$Q(s,\omega) = Q(s,\omega) + \alpha\left[r_t^{\tau} + \gamma^{\tau} \max_{\omega'} Q(s',\omega') - Q(s,\omega)\right]$$

- Onde:
  - τ: momento em que a option termina
  - r_t^{τ}: recompensa acumulada até a conclusão da option
- Converge para os valores Q ótimos

Option-critic

- Antes: options pré-definidas
- Options não precisam ser definidas manualmente
- **Option-critic** aprende **automaticamente** as options e quando usá-las ao invés de exigir sua definição manual



Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence (AAAI-17)

The Option-Critic Architecture

Pierre-Luc Bacon, Jean Harb, Doina Precup
Reasoning and Learning Lab, School of Computer Science
McGill University
{pbacon, jharb, dprecup}@cs.mcgill.ca

Abstract

Temporal abstraction is key to scaling up learning and planning in reinforcement learning. While planning with temporally extended actions is well understood, creating such abstractions autonomously from data has remained challenging. We tackle this problem in the framework of options [Sutton, Precup & Singh, 1999; Precup, 2000]. We derive policy gradient theorems for options and propose a new *option-critic* architecture capable of learning both the internal policies and the termination conditions of options, in tandem with the policy over options, and without the need to provide any additional rewards or subgoals. Experimental results in both discrete and continuous environments showcase the flexibility and efficiency of the framework.

Introduction

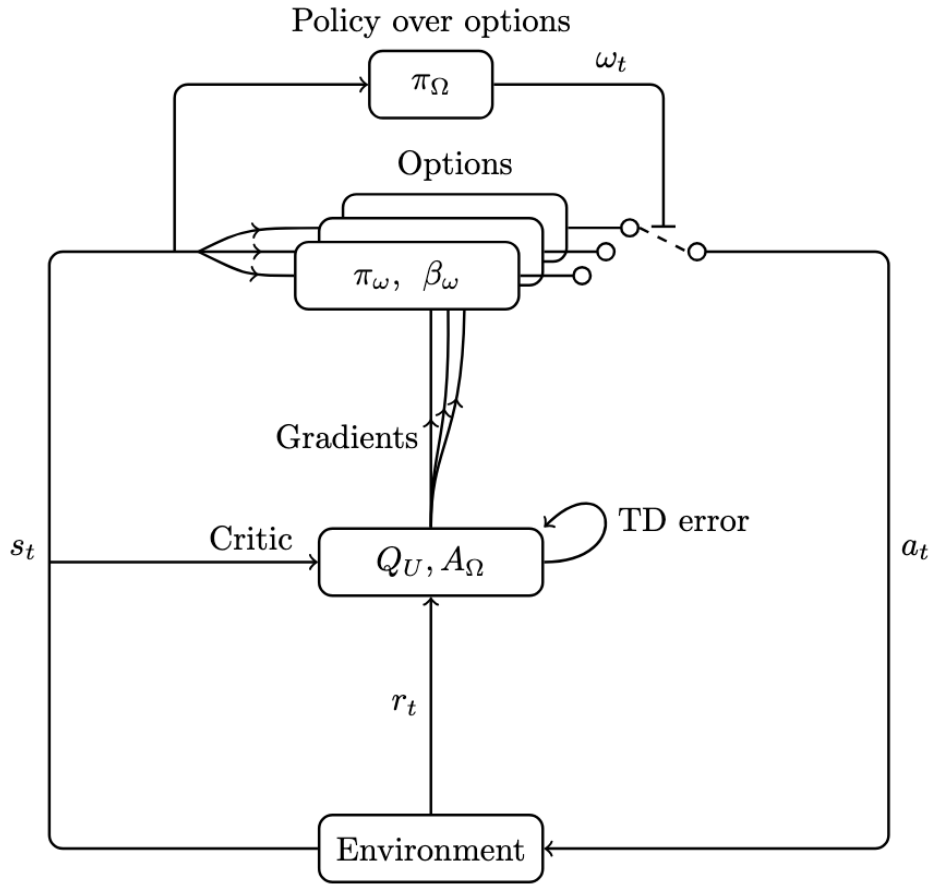
Temporal abstraction allows representing knowledge about courses of action that take place at different time scales. In reinforcement learning, *options* (Sutton, Precup, and Singh 1999; Precup 2000) provide a framework for defining such courses of action and for seamlessly learning and planning with them. *Discovering* temporal abstractions autonomously has been the subject of extensive research efforts in the last 15 years (McGovern and Barto 2001; Stolle and Precup 2002; Menache, Mannor, and Shimkin

ing options. Based on the policy gradient theorem (Sutton et al. 2000), we derive new results which enable a gradual learning process of the intra-option policies and termination functions, simultaneously with the policy over them. This approach works naturally with both linear and non-linear function approximators, under discrete or continuous state and action spaces. Existing methods for learning options are considerably slower when learning from a single task: much of the benefit comes from re-using the learned options in similar tasks. In contrast, we show that our approach is capable of successfully learning options within a single task without incurring any slowdown and while still providing benefits for transfer learning.

We start by reviewing background related to the two main ingredients of our work: policy gradient methods and options. We then describe the core ideas of our approach: the intra-option policy and termination gradient theorems. Additional technical details are included in the appendix. We present experimental results showing that our approach learns meaningful temporally extended behaviors in an effective manner. As opposed to other methods, we only need to specify the number of desired options; it is not necessary to have subgoals, extra rewards, demonstrations, multiple problems or any other special accommodations (however, the approach can take advantage of pseudo-reward functions

Option-critic

- O agente aprende **como** executar e **quando** terminar uma option
- Options disponíveis em <u>todos</u> os estados
- **Intra-Option policy gradient theorem**: permite otimizar as políticas internas das options
- **Termination gradient theorem**: permite otimizar suas condições de término



Variações do option-critic

Algoritmo	Problema abordado	Solução proposta	Referência
Deliberation Cost (DC)	Options terminam com muita frequência	Introduz um custo associado ao término das options	Harb et al. (2018)
Termination-Critic (TC)	Condições de término pouco significativas	Utiliza teoria da informação para aprender condições de término	Harutyunyan et al. (2019)
Interest Option-Critic (IOC)	Options disponíveis em todos os estados	Aprende funções de interesse que influenciam a seleção das options	Khetarpal et al. (2020)

Prática

Conclusão

Quiz

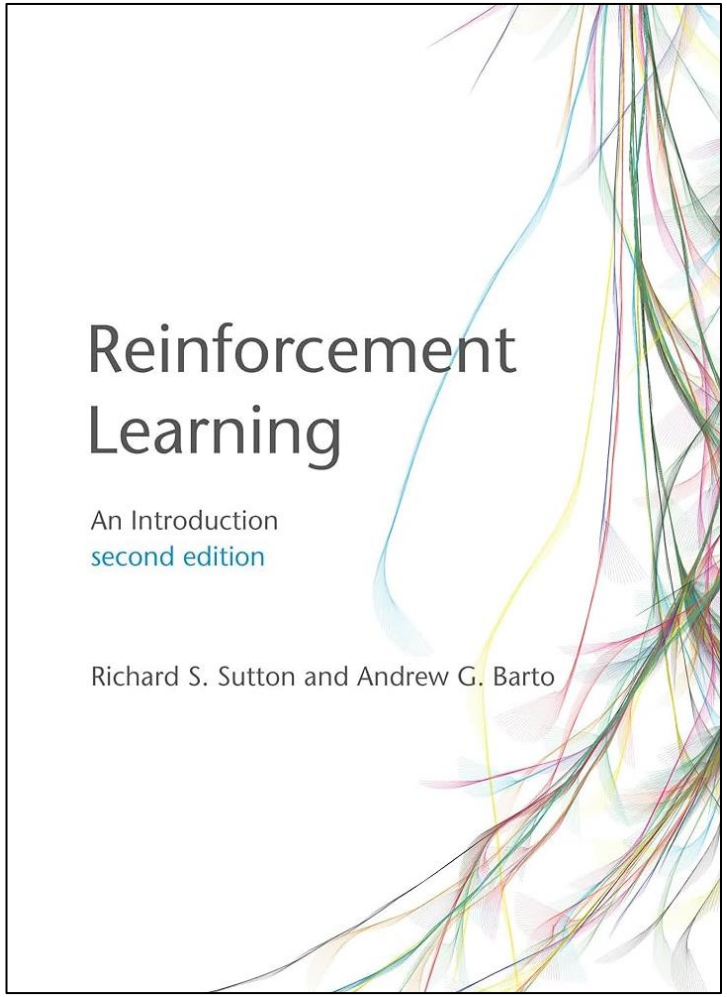
Quais afirmações sobre aprendizado por reforço são verdadeiras?

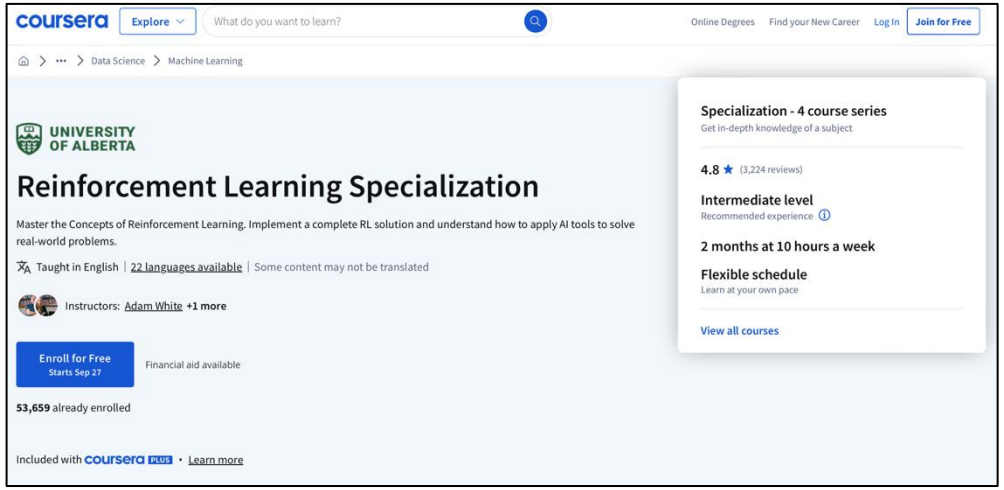
Take-home message

- **RL** lida com problemas de tomada de decisão sequencial
- **Recompensas** fornecem visão mais clara dos objetivos
- RL pode **generalizar** outros tipos de aprendizado
- **Options** lidam com abstração temporal e permitem reutilizar comportamentos

Para saber mais...

- Livro Sutton e Barto
- RL Specialization Coursera
- **Disciplina de RL Prof. Gabriel** (Unisinos)
- Gymnasium, stable-baselines3, options-envs (!), options-baselines (!), ...





Gymnasium



Parceiros


inst
kunumi


CNPq


University of
Massachusetts
Amherst


VUB
ARTIFICIAL
INTELLIGENCE
RESEARCH GROUP

UNISINOS

ERAMIA-RS
II ESCOLA REGIONAL DE APRENDIZADO DE MÁQUINA E INTELIGÊNCIA ARTIFICIAL DA REGIÃO SUL
2026

Obrigado!



Gabriel de O. Ramos
Arturo de Souza
Augusto A. F. Leal

UNISINOS

© 2026 by Gabriel de Oliveira Ramos under CC BY-NC-ND